

COURSE

UNIVERSITÉ TOULOUSE III

MASTER SID

Deep Learning

Author:

Emmanuelle Claeys

Date: September 30, 2024

September 30, 2024

Contents

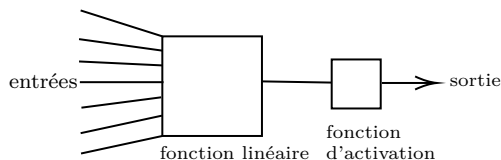
1	Reseau de neurones	2
1.1	Perceptron	2
1.1.1	Perceptron linéaire	2
1.2	Biais - Perceptron affine	4
1.3	Rappel sur la séparation linéaire	9
1.4	Vocabulaire	9
1.5	Réseau de neurones	10
1.5.1	Couche de neurones	10
2	Théorème d'approximation universelle	12
2.1	Ensemble réalisable	12
2.2	Approximation d'ensembles	15
2.3	Théorème d'approximation universelle	16
2.4	Fonction Heaviside	16
2.5	Fonction Créneaux	17
2.6	Fonction en escalier	18
2.7	Théorème d'approximation universelle (une variable)	19
2.8	Théorème d'approximation universelle (deux variables et plus)	20
3	Gradient	20
3.1	Dérivée partielle	20
3.2	Gradient	21
3.3	Gradient pour un réseau de neurones	21
3.3.1	Formule du gradient	22
3.3.2	Exemple	24
3.3.3	Second exemple	25
4	Descente de gradient	26
4.1	Descente de gradient classique	27
4.2	Rappel sur les lignes de niveaux (fonction à plusieurs variables)	27
4.2.1	Où est le minimum ?	27
4.2.2	Algorithme du gradient	29
4.3	Optimisation	30
4.3.1	Faire varier le pas	30
4.3.2	Régression linéaire $y = ax + b$	31
4.3.3	Régression linéaire et neurone	32
4.4	Descente de gradient stochastique	33
4.4.1	Petits pas à petits pas	33
4.5	Différentes fonctions d'erreurs	36
4.6	Descente par lots	36

1 Réseau de neurones

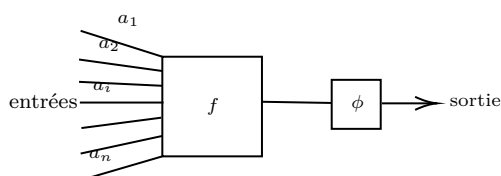
1.1 Perceptron

1.1.1 Perceptron linéaire

Le principe du perceptron linéaire est de prendre des valeurs en entrées, de faire un calcul simple et de renvoyer une valeur de sortie. Les calculs dépendent de paramètres propres de chaque perceptron.



Le calcul effectué par un perceptron se décompose en deux phases : un calcul par une fonction linéaire f , suivi d'une fonction d'activation ϕ .

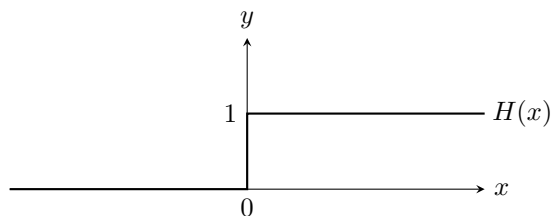


Détaillons chaque phase.

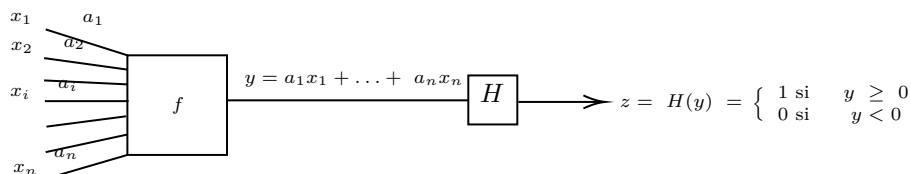
- **Partie linéaire.** Le perceptron est d'abord muni de poids $a_1, \dots, a_n$ qui déterminent une fonction linéaire.
- **Fonction d'activation.** La valeur renvoyée par la fonction linéaire f est ensuite composée par une fonction d'activation ϕ .
- **Sortie.** La valeur de sortie est donc $\phi(a_1x_1 + a_2x_2 + \dots + a_nx_n)$.

Dans ce chapitre, la fonction d'activation sera presque toujours la fonction marche de Heaviside :

$$\begin{cases} H(x) = 1 & \text{si } x \geq 0, \\ H(x) = 0 & \text{si } x < 0, \end{cases}$$



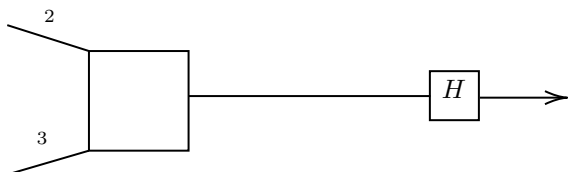
Voici ce que fait un perceptron linéaire de poids $a_1, \dots, a_n$ et de fonction d'activation la fonction marche de Heaviside :



On peut maintenant définir ce qu'est un perceptron. Un **perceptron linéaire** à n variables couplé à une fonction de Heaviside est une fonction linéaire à n coefficients réels $a_1, \dots, a_n$ auxquels est associé la fonction $F : \mathbb{R} \rightarrow \mathbb{R}$ définie par $F \circ f$, c'est-à-dire :

$$\begin{cases} F(x_1, \dots, x_n) = 1 & \text{si } a_1x_1 + \dots + a_nx_n \geq 0, \\ F(x_1, \dots, x_n) = 0 & \text{sinon} \end{cases}$$

Exemple : Voici un perceptron à deux entrées. Il est défini par les coefficients $a_1 = 2$ et $a_2 = 3$.

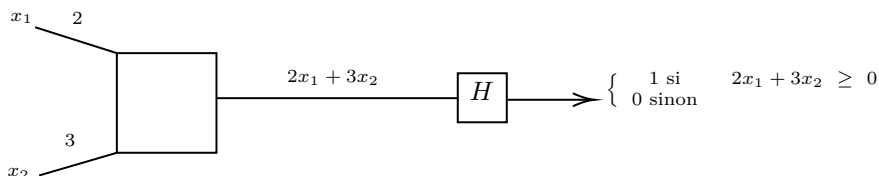


Notons x_1 et x_2 les deux réels en entrée. La fonction linéaire f est donc :

$$f(x_1, x_2) = 2x_1 + 3x_2$$

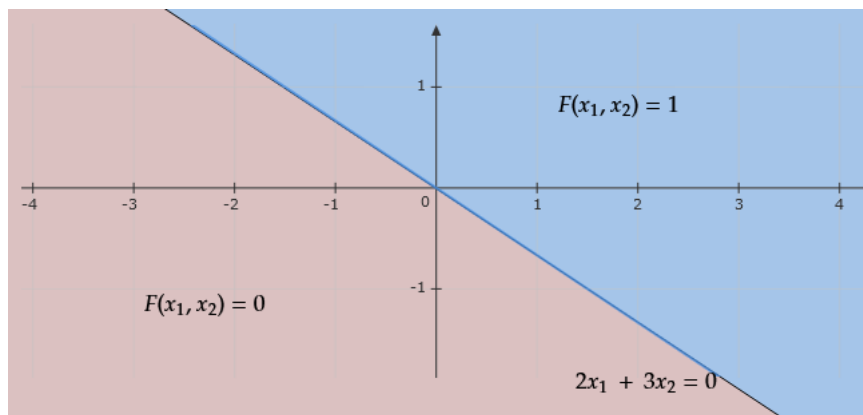
La valeur en sortie est donc :

$$\begin{cases} F(x_1, x_2) = 1 & \text{si } 2x_1 + 3x_2 \geq 0, \\ F(x_1, x_2) = 0 & \text{sinon} \end{cases}$$

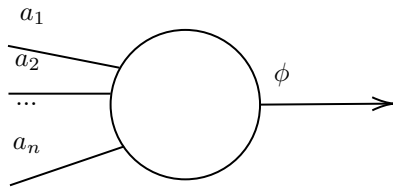


Exercice : Que renvoie le perceptron pour la valeur d'entrée $(x_1, x_2) = (4, -1)$?

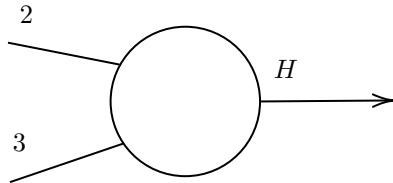
La fonction F prend seulement deux valeurs : 0 ou 1. La frontière correspond aux points (x_1, x_2) tels que $f(x_1, x_2) = 0$, c'est-à-dire à la droite $2x_1 + 3x_2 = 0$. Pour les points au dessus de la droite (ou sur la droite) la fonction F prend la valeur 1; pour les points en dessous de la droite, la fonction F vaut 0.



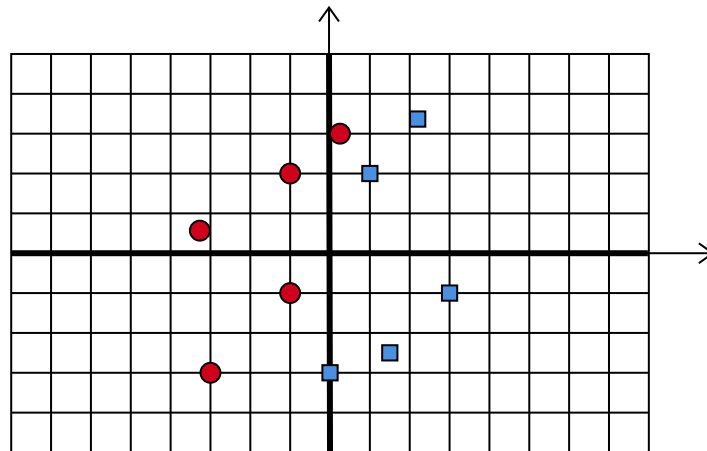
Nous représentons un perceptron par une forme plus condensée : sous la forme d'un **neurone** avec des poids sur les arrêtes d'entrées. Nous précisons également la fonction d'activation utilisée ϕ .



Voici le neurone à deux variables de l'exemple précédent.



Exemple Voici deux catégories de points : des ronds rouge et des carrés bleus. Comment trouver un perceptron qui les sépare ?



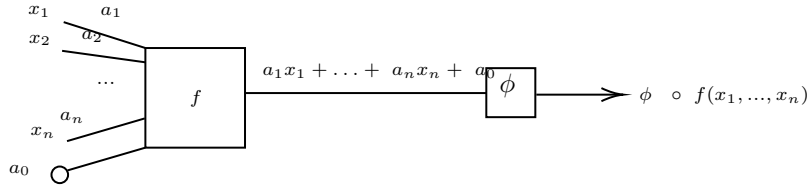
Exercice : Trouver deux poids a_1 et a_2 d'un perceptron, dont la fonction associée F vérifie $F(x_1, x_2) = 1$ pour les coordonnées des carrés et $F(x_1, x_2) = 0$ pour les ronds. Hachurez les zones d'activation et représenter le perceptron par un graph.

1.2 Biais - Perceptron affine

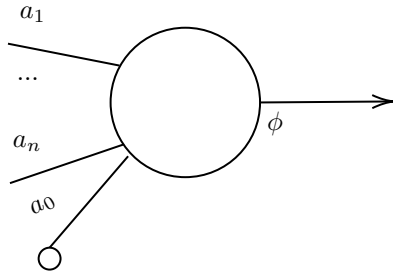
Pour l'instant notre perceptron à deux entrées sépare le plan en deux parties selon une droite passant par l'origine. Nous allons plus loin avec le perceptron affine. On modifie la définition du perceptron, sans changer le nombre d'entrées, mais en ajoutant un poids supplémentaire. Dans le cas de n entrées il y a donc $n + 1$ poids :

- les **coefficients** $a_1, \dots, a_n \in \mathbb{R}$
- et le **biais** $a_0 \in \mathbb{R}$ qui définit la fonction affine :

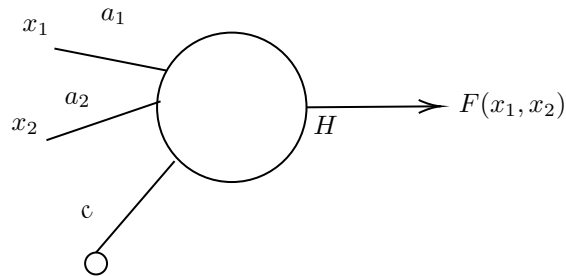
$$f(x_1, \dots, x_n) = a_1x_1 + a_2x_2 + \dots + a_nx_n + a_0$$



On représente le neurones avec une nouvelle arête, pondérée par le biais. L'arête est terminée par un rond pour signifier que cela ne correspond pas à une entrée.



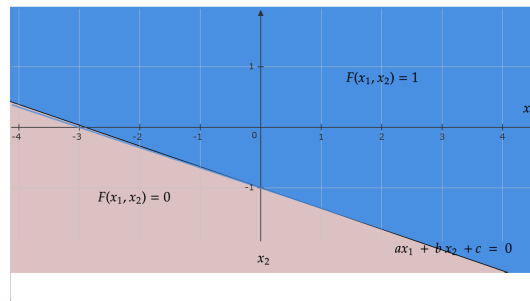
Dans le cas de deux entrées, les poids sont trois réels a_1 et a_2 (les coefficients) et c le biais.

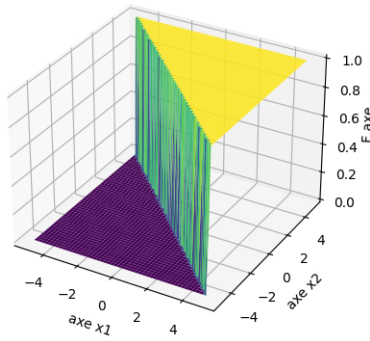


La fonction $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ est la fonction affine $f(x_1, x_2) = a_1x_1 + a_2x_2 + c$. Si la fonction d'activation est la fonction marche de Heaviside, alors le perceptron affine définit une fonction $F : \mathbb{R}^2 \rightarrow \mathbb{R}$

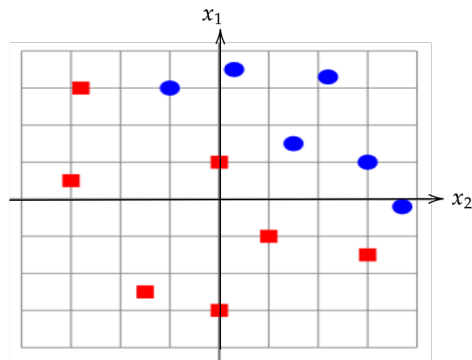
$$\begin{cases} F(x_1, x_2) = 1 & \text{si } ax_1 + a_2x_2 + c \geq 0, \\ F(x_1, x_2) = 0 & \text{sinon} \end{cases}$$

Un perceptron affine à deux entrées sépare donc le plan en deux parties, la frontière étant la droite d'équation $ax_1 + bx_2 + c = 0$. D'un côté de cette droite la fonction F vaut 1, de l'autre elle vaut 0.

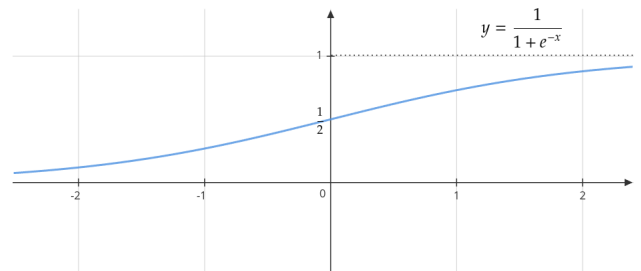
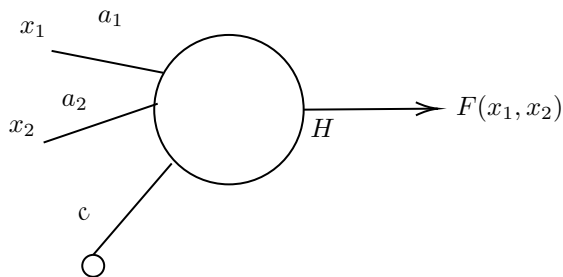


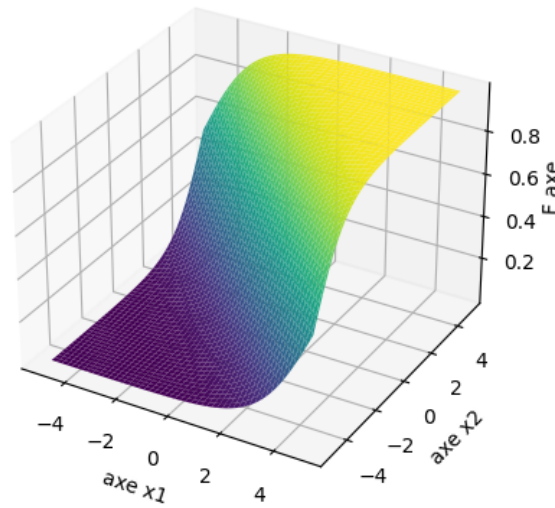


Exercice : Trouver un perceptron qui distingue les carrés des ronds.

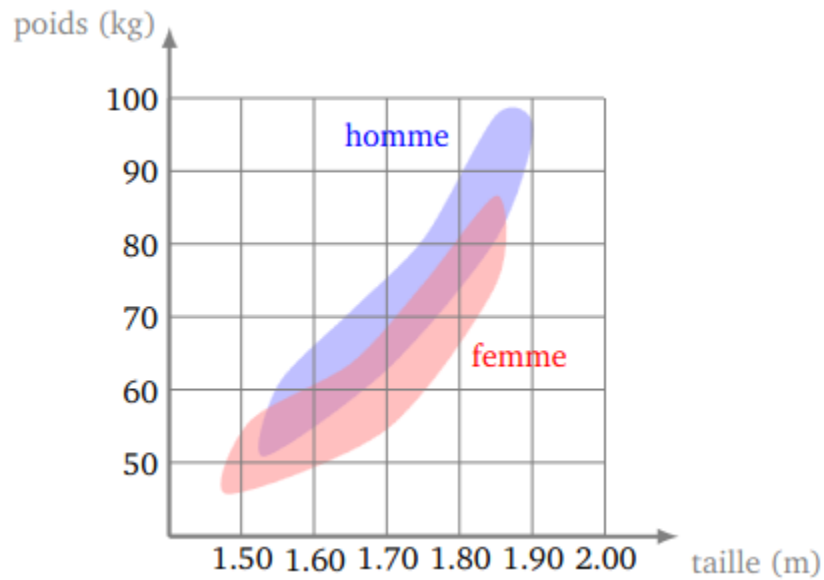


Il n'est pas toujours possible de répondre à une question par "oui" ou "non". Pour y remédier on peut changer de fonction d'activation en utilisant par exemple la fonction sigmoïde σ . Dans ce cas, à chaque point du plan, on associe, non plus 0 ou 1, mais une valeur entre 0 et 1. Ce nombre peut correspondre à un degré de certitude. Par exemple avec une question "est-ce que cette photo est un chat ?", si la sortie vaut 0.8, cela signifie "c'est un chat à 80%". Si la sortie vaut 0.1 alors ce n'est pas un chat avec une probabilité de $1-0.1$ c'est à dire 90%. Voici un neurone à deux entrées muni de la fonction d'activation sigmoïde définie par $\sigma(x) = \frac{1}{1+e^{-x}}$.



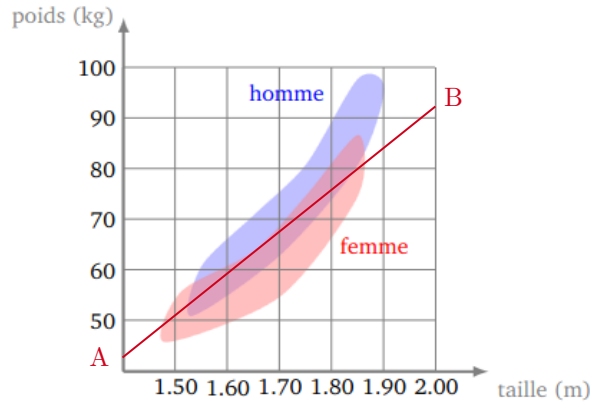


Voici les données (fictives) de la répartition des hommes et des femmes selon leur taille et leur poids.



Problème : la taille et le poids d'une personne étant donnés, trouver un perceptron qui répond à la question "cette personne est-elle un homme ou une femme" ? Au vu de la superposition des zones, il n'est pas possible de répondre avec certitude. On construit donc un perceptron selon les idées suivantes :

- on trace une droite qui sépare au mieux les hommes des femmes, par exemple ici la droite qui passe par les points A(1.40,42) et B(2,93). Pour trouver la droite d'équation associée à ces deux points on fait $a = \frac{y_B - y_A}{x_B - x_A} = 85$ et comme $42 = 1.40 \times 85 + b$ on trouve $b = 42 - 1.40 \times 85 = -77$ (on peut faire pareil avec le point B. On obtient la droite d'équation $y = 85 \times x - 77$

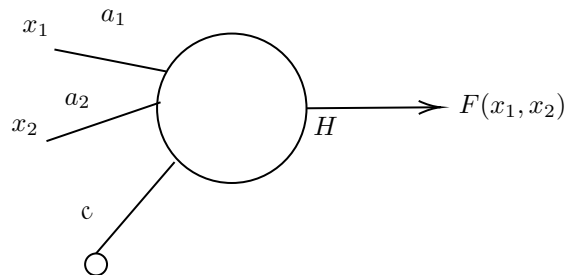


L'objectif est ensuite pour un couple de données 'taille' et 'poids' (soit x_1, x_2), de savoir si l'on est au dessus de la droite ou pas. Cela revient à dire que pour tout couple (x_1, x_2) :

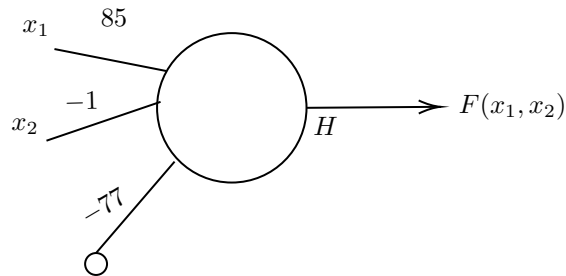
- Si $a \times x_1 + b - x_2 < 0$ alors la valeur de x_2 est plus grande que la valeur de la droite : on est au dessus à la droite (plutôt un homme donc)
- Si $a \times x_1 + b - x_2 > 0$ alors la valeur de x_2 est plus petite que la valeur de la droite : on est dessous de la droite (plutôt un femme donc)
- Si $a \times x_1 + b - x_2 = 0$ alors on est non binaire (ou alors on ne sait pas !)

Exercice : Dire si vous êtes au-dessus de la droite pour les couples $(1.77, 80); (1.50, 45); (1.77, 75)$.

On souhaite représenter notre preceptron avec la fonction Heaviside :



Pour regarder les valeurs de a_1 , a_2 et c on regarde dans notre équation $85 \times x_1 - 77 - x_2$ les coefficients associés à x_1 , x_2 et la constante. On trouve :



Mais plutôt que d'avoir une sortie 0,1 associée à la classe (homme-femme) on souhaite avoir une valeur probabiliste. Il faut donc utiliser une fonction sigmoïde qu'une fonction Heaviside. On applique donc la fonction $\sigma(x_1, x_2) = \frac{1}{1+e^{-(a_1 x_1 + a_2 x_2 + c)}}$ à notre couple (x_1, x_2) . Calculons la sortie du couple $(1.77, 75)$:

$\frac{1}{1+e^{-(85 \times 1.77 - 1 \times 75 - 77)}}$ avec ce lien. On trouve $F(1.77, 75) = \sigma(-1.55) \simeq 0.17$. Selon notre modèle cette personne est probablement un homme (car la valeur de F est < 0.5), plus précisément le modèle pense que c'est un homme avec une probabilité de $1 - 0.17 = 83\%$ et une femme avec une probabilité de 17% .

Exercice : Dire les probabilités respectives pour les couples $(1.77, 80); (1.50, 45); (1.67, 64)$.

Dans notre exemple, on peut discuter de la pertinence de la frontière car la séparation par une droite ne semble pas la mieux adaptée. On constate en effet que le poids varie en fonction du carré de la taille et une parabole serait plus adaptée qu'une droite pour faire cette séparation.

Un **perceptron affine** à n entrées est donc la données de $n+1$ poids $a_0, a_1, \dots, a_n \in \mathbb{R}$ et d'une fonction d'activation ϕ qui définissent une fonction $F : \mathbb{R}^n \rightarrow \mathbb{R}$ par la formule :

$$F(x_1, \dots, x_n) = \phi(a_1 x_1 + \dots + a_n x_n + a_0)$$

1.3 Rappel sur la séparation linéaire

Une **droite** du plan $\mathbb{R}^2$ est l'ensemble des points (x, y) vérifiant une équation du type $ax + by + c = 0$. Un **plan** de l'espace $\mathbb{R}^3$ est l'ensemble des points (x, y, z) vérifiant une équation du type $ax + by + cz + d = 0$. Nous généralisons cette notion en dimension plus grande.

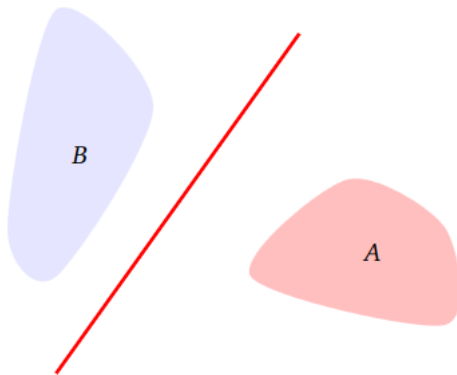
Un hyperplan (affine) de $\mathbb{R}^n$ est l'ensemble des points $(x_1, \dots, x_n)$ qui vérifient une équation du type :

$$a_1 x_1 + a_2 x_2 + \dots + a_n x_n + a_0 = 0$$

où $a_1, \dots, a_n$ sont des réels (non tous nuls) et a_0 est aussi un réel. Le but d'un perceptron affine est de séparer deux ensembles. Par exemple deux ensembles A et B du plan seront séparés par une droite, deux ensemble de l'espace sont séparés par un plan.

Deux ensembles A et B sont linéairement séparables s'il existe un hyperplan qui sépare A et B .

Plus précisément, s'il existe des réels $a_0, a_1, \dots, a_n$ tels que $a_1 x_1 + a_2 x_2 + \dots + a_n x_n \geq 0$ pour tout $(x_1, \dots, x_n) \in A$ et $a_1 x_1 + \dots + a_n x_n + a_0 < 0$ pour tout $(x_1, \dots, x_n) \in B$.

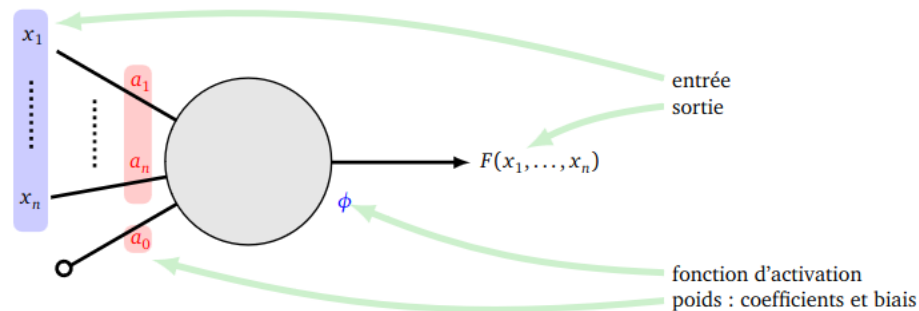


Deux ensembles A et B de $\mathbb{R}^n$ sont linéairement séparables si et seulement si, il existe un perceptron affine dont la fonction F vaut 1 sur A et 0 sur B .

1.4 Vocabulaire

Résumons le vocabulaire utilisé ci-dessus ainsi que les termes anglais correspondant :

- Le **perceptron linéaire** ou **neurone artificiel**, a un biais qui vaut 0, à la différence du **perceptron affine** pour lequel le biais est un réel quelconque;
- Les **poids** (weights) ou **paramètres** sont les **coefficients** $a_1, \dots, a_n$ auxquels s'ajoute le **biais** (*bias*) (dans ce cours, on considère le biais comme un poids mais ce n'est pas toujours le cas dans la littérature).
- Un **perceptron** est la combinaison des poids et d'une fonction d'activation,
- Chaque perceptron définit une fonction F qui est la composée d'une **fonction affine** f et d'une **fonction d'activation** ϕ ; dans ce chapitre nous avons utilisé la fonction Heaviside (*step function*) et la fonction sigmoïde.
- On utilise un perceptron pour prédire une sortie (output) $F(x_1, \dots, x_n) \in \mathbb{R}$ (1 ou 0 si on utilise une Heaviside par exemple) à partir d'une **entrée** (input) $x_1, \dots, x_n \in \mathbb{R}^n$.

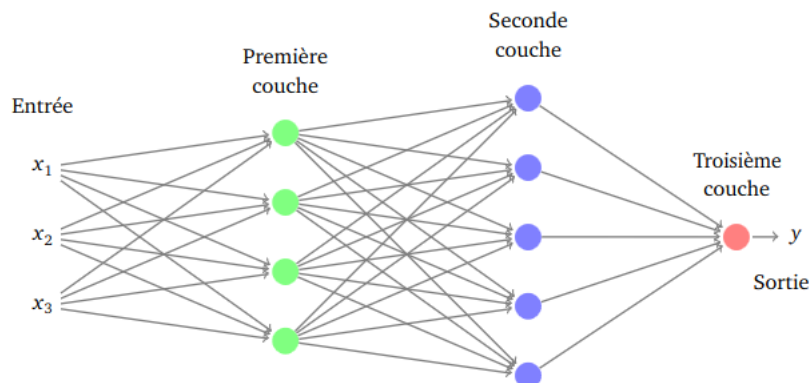


1.5 Réseau de neurones

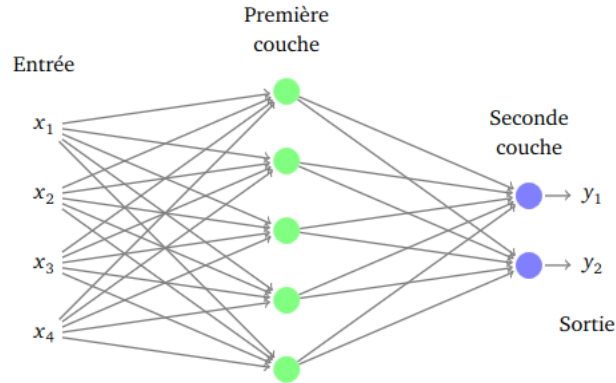
Un neurone permet de séparer l'espace en deux parties, la frontière étant linéaire (avec une entrée en dimension 2 c'est une droite, avec 3 c'est un plan). Un neurone est trop élémentaire pour résoudre nos problèmes. Par exemple il ne peut pas réaliser le "ou exclusif". Comment faire mieux ? En connectant plusieurs neurones !

1.5.1 Couche de neurones

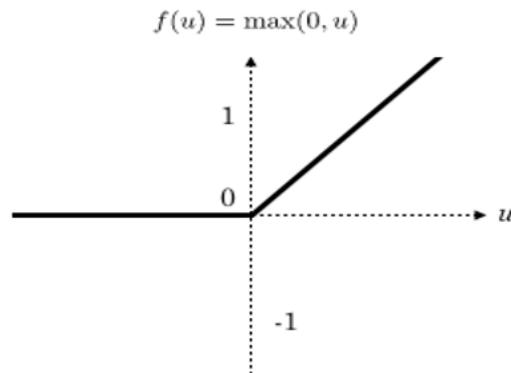
Un **réseau de neurones** est la juxtaposition de plusieurs neurones, regroupés par **couches**.



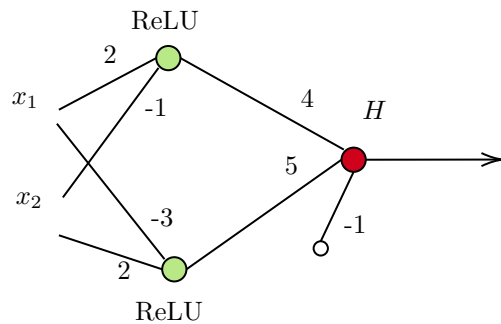
À un réseau de neurones on associe une fonction. Si à la dernière couche, la fonction ne contient qu'un seul neurone (voir ci-dessus) cette fonction est $F : \mathbb{R}^n \rightarrow \mathbb{R}$, $y = F(x_1, \dots, x_n)$ où $(x_1, \dots, x_n)$ est l'**entrée** et y est la **sortie**. Sinon (voir ci-dessous), la fonction est $F : \mathbb{R}^n \rightarrow \mathbb{R}^p$, $y = F(x_1, \dots, x_n)$ où $(x_1, \dots, x_n)$ est l'**entrée** et $(y_1, \dots, y_p)$ est la **sortie**.



Exemple : Voici un réseau de neurones à deux couches : 2 neurones (perceptrons linéaires) sur la première couche (ayant pour fonction d'activation la fonction ReLU), qui correspond à

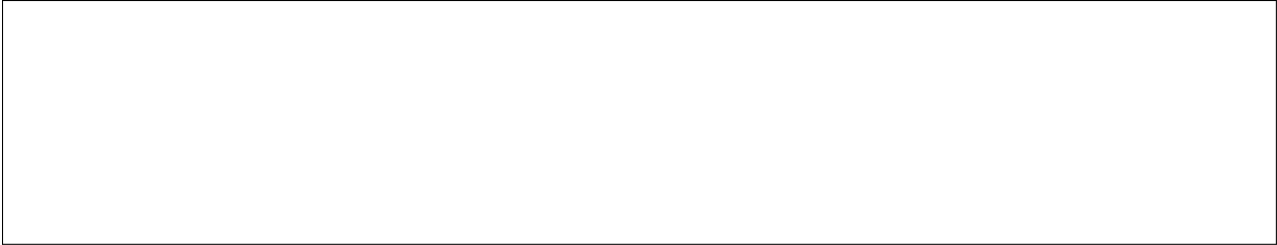


et 1 neurone (perceptron affine) sur la seconde couche (ayant pour fonction d'activation la fonction Heaviside) :

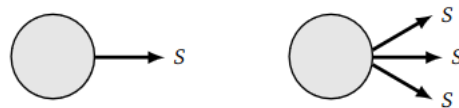


Regardons pour $x_1 = 4$ et $x_2 = 7$. On commence par calculer la sortie de chaque neurone de la première couche. Ces sorties sont les entrées du neurone de la seconde couche. Pour le premier neurone, on effectue le calcul $2 \times 4 + (-1) \times 7 = 1 \geq 0$, le réel étant positif, la fonction ReLU le garde inchangé : le premier neurone renvoie la valeur $s_1 = 1$. Pour le second neurone, on calcul $(-3) \times 4 + 2 \times 7 = 2 \geq 0$ et revoit $s_2 = 2$. Le neurone de la couche de sortie reçoit en entrées s_1 et s_2 et effectue le calcul $4 \times 1 + 5 \times 2 - 1 = 13 \geq 0$. La fonction d'activation étant H , ce neurone renvoie 1. Ainsi $F(4, 7) = 1$.

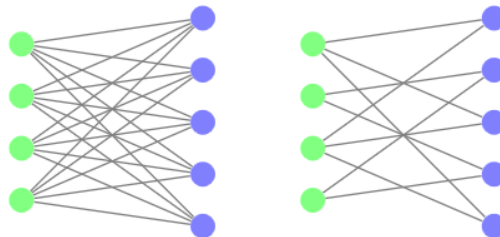
Exercice : Donner les sorties de $(3, 2)$ et $(\frac{1}{10}, \frac{1}{10})$.



- Chaque neurone est défini par ses poids et une fonction d'activation. Pour une couche donnée, on choisit toujours la même fonction d'activation. En général on choisit la même fonction d'activation pour tout le réseau, sauf peut-être pour la couche de sortie.
- Un neurone prend plusieurs valeurs en entrée mais ne renvoie qu'une seule valeur en sortie. Si ce neurone est connecté à plusieurs autres neurones, il envoie la même valeur à tous.



- On peut relier un neurone à tous les neurones de la couche suivante (voir ci-dessous, figure de gauche). On dit que la seconde couche est **dense** ou **complètement connectée**. Mais on peut aussi choisir de ne relier que certains neurones entre eux (voir ci-dessous figure de droite). S'il n'y a pas d'arêtes du neurone A vers le neurone B, c'est que la sortie de A n'intervient pas comme entrée de B (cela revient à imposer un poids nul entre ces deux neurones).

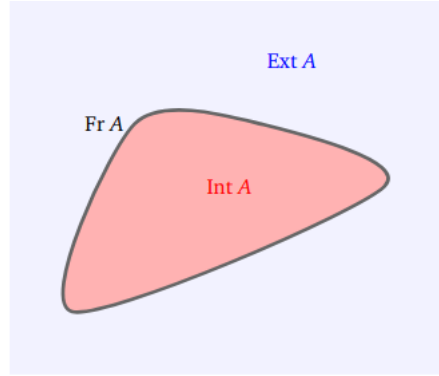


2 Théorème d'approximation universelle

2.1 Ensemble réalisable

Nous aimerions savoir quels ensembles peuvent être décrits par un réseau de neurones. On rappelle qu'un ensemble $A \in \mathbb{R}^2$ découpe le plan en trois parties disjointes : intérieur, frontière, extérieur :

$$Int(A) \quad Fr(A) \quad Ext(A)$$

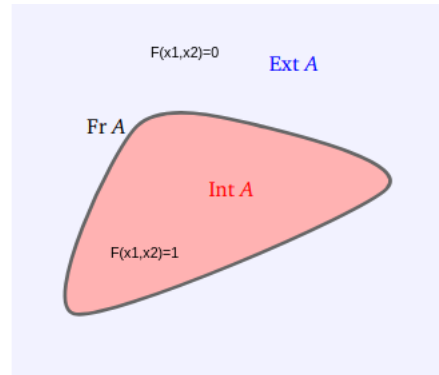


Un ensemble A est dit **réalisable par un réseau de neurones** s'il existe un réseau de neurones $\mathcal{R}$ de fonctions d'activation $\phi : H$ tel que la fonction $F : \mathbb{R}^2 \rightarrow \mathbb{R}$ associée à $\mathcal{R}$ vérifiant

$$F(x_1, x_2) = 1 \quad \text{pour tout } (x_1, x_2) \in \text{Int}(A)$$

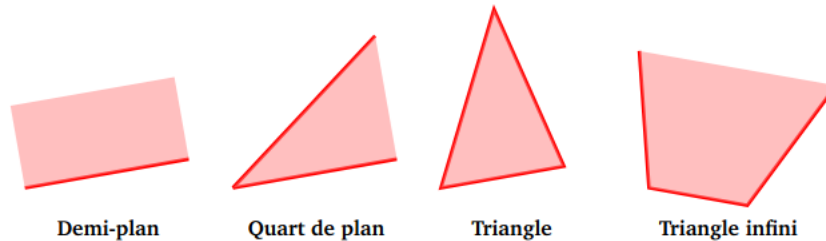
et

$$F(x_1, x_2) = 0 \quad \text{pour tout } (x_1, x_2) \in \text{Ext}(A)$$



Remarque : on n'exige rien sur la frontière $\text{Fr}(A)$, F peut y prendre la valeur 0 ou la valeur 1. Voici les types d'ensembles que nous avons déjà réalisés :

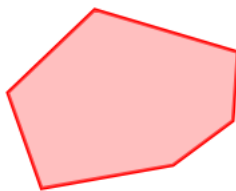
- les demi-plans,
- les "quart de plans",
- les zones triangulaires,
- les triangles avec un sommet "à l'infini"



En augmentant le nombre de neurones, on peut réaliser les quadrilatères convexes et plus généralement n'importe quelle zone polygonale convexe¹.

¹Un polygone non convexe (voir aussi non-convexe), concave ou rentrant, désigne un polygone simple ayant au moins un angle rentrant intérieur

Proposition : Toutes zones polygonales convexes à n côtés sont réalisables par un réseau de $n + 1$ neurones

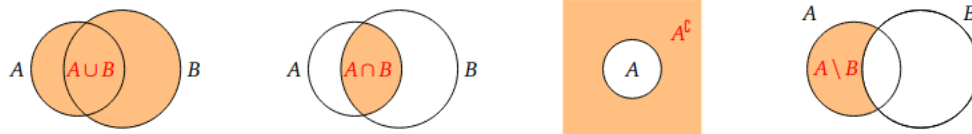


Polygone convexe

Démonstration Un polygone convexe à n côtés est l'intersection de n demi-plans. Chaque demi-plan, bordé par une droite d'équation du type $ax + by + c = 0$, correspond à un neurone de la première couche dont les coefficients sont (a, b) et le biais est c (ou bien $(-a, -b)$ et $-c$). Sur la seconde couche, on place un neurone qui réalise l'opération "ET" sur les n entrées : tous ses coefficients sont 1 et le biais est $-n$. On peut également approximer des opérations élémentaires sur des ensembles réalisables par des réseaux de neurones :

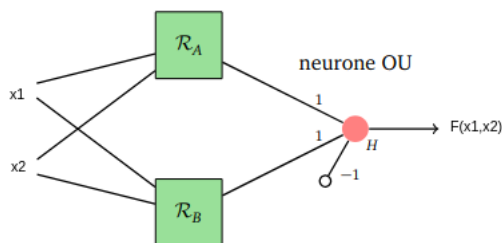
$$A \cup B \quad A \cap B \quad A^C \quad A \setminus B$$

sont aussi des ensembles réalisables.

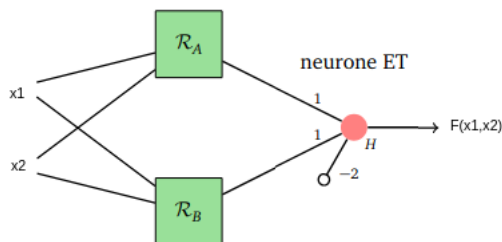


Démonstration

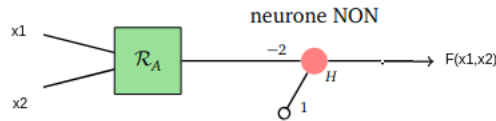
- Si A est réalisable par un réseau $\mathcal{R}_A$ et B par un réseau $\mathcal{R}_B$ alors on crée un nouveau réseau $\mathcal{R}$ en superposant $\mathcal{R}_A$ et $\mathcal{R}_B$. Ainsi si (x_1, x_2) est dans $A \cup B$ il est dans A ou dans B , une des sorties $\mathcal{R}_A$ ou $\mathcal{R}_B$ vaut alors 1 et le neurone sortant de $\mathcal{R}$ s'active.



- Pour réaliser $A \cap B$, on remplace le neurone "OU" par un neurone "ET"



- Pour réaliser le complément d'un ensemble A , on utilise l'opération "non" : 0 s'envoie sur 1 et 1 sur 0. Ceci se fait par l'application $s \rightarrow H(1 - 2s)$. Il faut rajouter un neurone "NON" à $\mathcal{R}$.



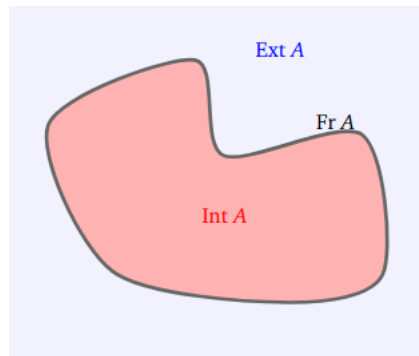
Exercice : Faire la démonstration et représenter le réseau associé à $A \setminus B$.

Proposition Tout polygone (convexe ou non) est réalisable par un réseau de neurones.

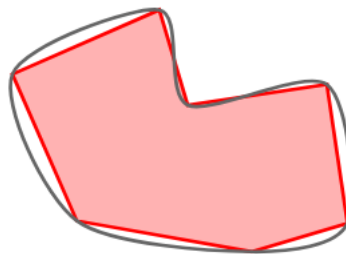
Démonstration. Un polygone peut être découpé en triangles. Chaque triangle est réalisable, donc l'union des triangles l'est aussi.

2.2 Approximation d'ensembles

On rappelle qu'une **courbe de Jordan** C est une courbe fermée simple (c'est l'image d'un cercle par une application continue injective). Le théorème suivant est une sorte de théorème d'approximation universelle géométrique.



Théorème Un ensemble A délimité par une courbe de Jordan peut être approché d'autant près que l'on veut par un ensemble A' réalisable par un réseau de neurones. Il s'agit juste du fait qu'une courbe de Jordan peut être approchée par un polygone. Ce résultat théorique ne nous dit cependant pas comment choisir la structure du réseau ou les poids.



2.3 Théorème d'approximation universelle

Nous allons maintenant prouver qu'un réseau de neurones bien construit peut approcher n'importe quelle fonction. Dans cette section nous partons d'une seule entrée $x \in \mathbb{R}$ avec une seule sortie $y = F(x) \in \mathbb{R}$. Les réseaux de neurones de cette section produisent donc des fonctions $F : \mathbb{R} \rightarrow \mathbb{R}$.

L'objectif est le suivant : on nous donne une fonction $f : [a, b] \rightarrow \mathbb{R}$ et nous devons trouver un réseau, tel que la fonction F associée à ce réseau soit proche de f :

$$F(x) \approx f(x) \quad \text{pour tout} \quad x \in [a, b]$$

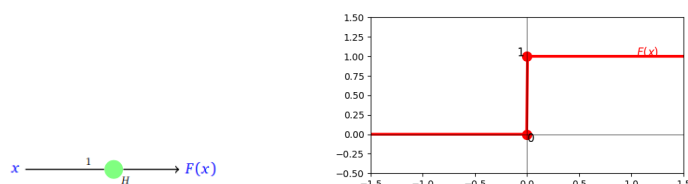
Pour paramétrer le réseau nous allons bien sûr fixer des poids mais avant cela choisissons les fonction d'activation :

- pour le neurone en sortie, on choisit la fonction identité $\phi(x) = x$,
- pour tous les autres neurones, on choisit la fonction Heaviside.

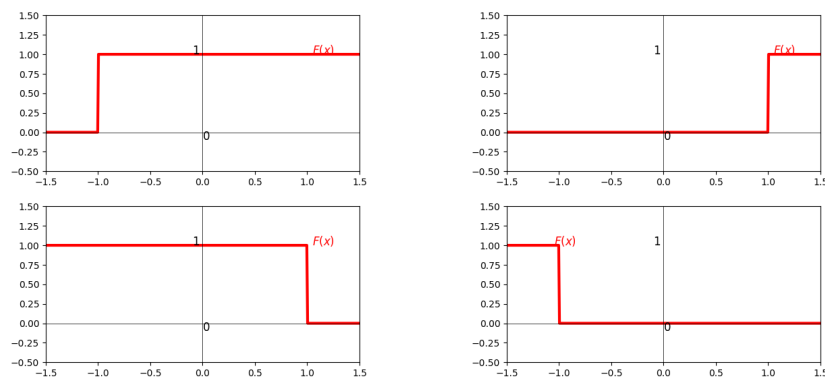
2.4 Fonction Heaviside

Nous allons réaliser des fonctions de plus en plus compliquées à partir d'éléments très simples. Commençons par étudier le comportement d'un seul neurone avec la fonction d'activation Heaviside (on rajoutera le neurone de sortie plus tard). Prenons la fonction standard ($a = 1, b = 1$), le code est disponible sur ce lien

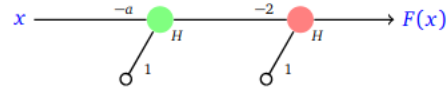
```
def heaviside_fun(a, b, x):
    return np.where(a * x + b >= 0, 1, 0)
```



Exercice : Représenter les perceptron et modifier le code python pour obtenir les sorties associées au graphiques suivants. Commentez l'évolution de la courbe en fonction du signe de a et b .

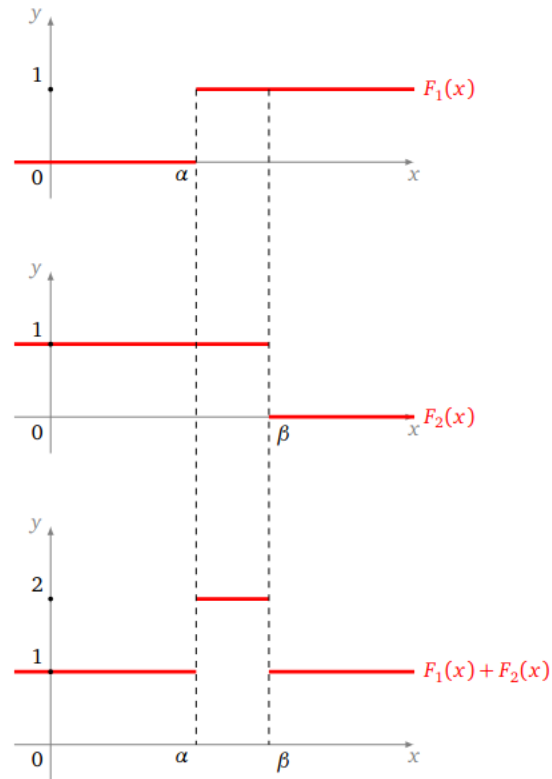


Exercice : Représenter le graphique associé au perceptron suivant.

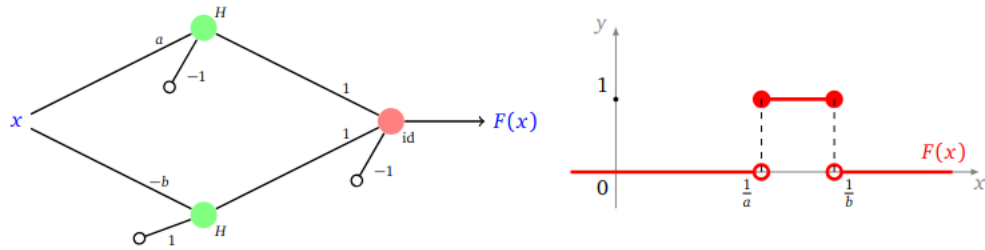


2.5 Fonction Créneaux

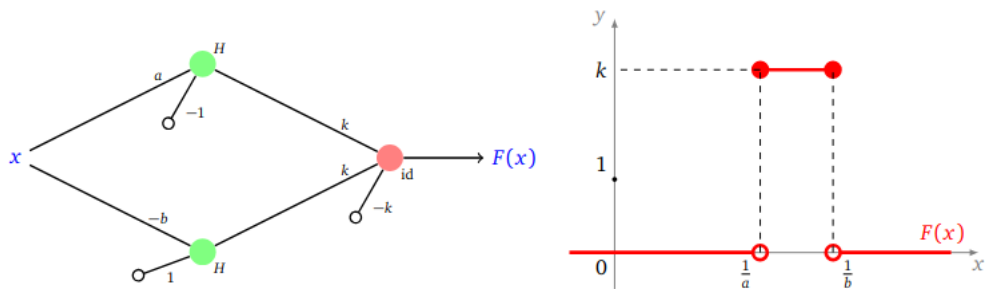
Pour réaliser un "créneau", l'idée est d'additionner une marche à l'endroit et une marche à l'envers.



Pour effectuer cette opération, nous allons construire un réseau avec deux neurones sur la première couche (de fonction d'activation H) et un neurone sur la seconde couche (de fonction d'activation identité) qui additionne les deux sorties précédentes et soustrait 1 (afin de ramener la ligne de base à 0).

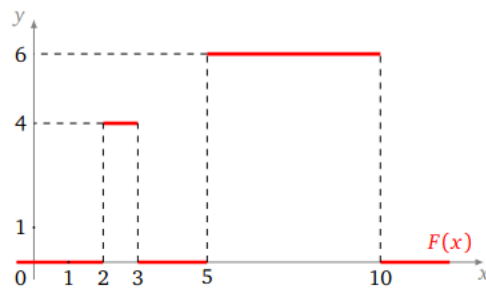


Si on veut une marche plus haute il suffit de changer les poids du neurone de sortie d'un facteur k

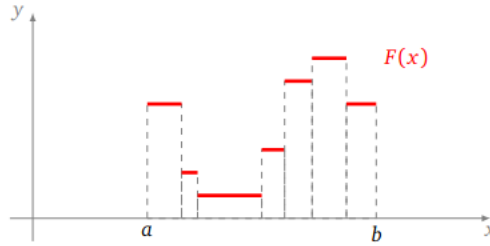


2.6 Fonction en escalier

Exercice : Représenter le perceptron associé à cette fonction et calculer la valeur de la fonction F .



Remarque : il peut y avoir un problème pour réaliser deux créneaux contigus. Si on n'y prend pas garde, la valeur est mauvaise à la jonction (c'est la somme des deux valeurs).
Une **fonction en escalier** est une fonction qui est constante sur un nombre fini d'intervalles bornés.



Proposition : Toute fonction en escalier est réalisable par un réseau de neurones. *Preuve* : Soit n le nombre de marches de l'escalier. On construit un réseau de $2n + 1$ neurones. La première couche est constituée de n paires de neurones, chaque paire réalise une marche de l'escalier. La seconde couche contient uniquement le neurone de sortie, les coefficients sont choisis pour ajuster la hauteur de la marche et le biais assure que la fonction vaut 0 en dehors de l'escalier. Si on veut les valeurs exactes aux bornes des marches, il faut éventuellement ajouter des neurones de type « NON » entre la première couche et le neurone de sortie.

2.7 Théorème d'approximation universelle (une variable)

Nous pouvons maintenant énoncer le résultat théorique le plus important de ce chapitre.

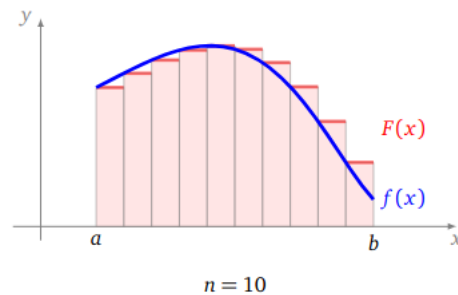
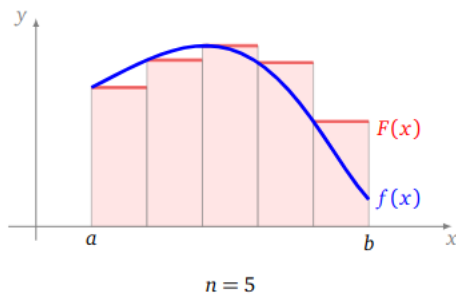
Théorème d'approximation universelle : Toute fonction continue $f : [a, b] \rightarrow \mathbb{R}$ peut être approchée d'aussi près que l'on veut par une fonction $F : [a, b] \rightarrow \mathbb{R}$ réalisée par un réseau de neurones.

Plusieurs commentaires importants.

- Tout d'abord rappelons que nous réalisons nos neurones avec des fonctions d'activation H (marche de Heaviside) sauf pour le neurone de sortie qui a pour fonction d'activation l'identité.
- Les hypothèses sont importantes : f est continue et définie sur un intervalle fermé et borné. Si une des hypothèses venait à manquer l'énoncé serait faux.
- Que signifie « approcher d'aussi près que l'on veut la fonction f » ? C'est dire que pour chaque $\epsilon > 0$, il existe une fonction F (ici issue d'un réseau de neurones), telle que :

$$\text{pour tout } x \in [a, b] |f(x) - F(x)| < \epsilon$$

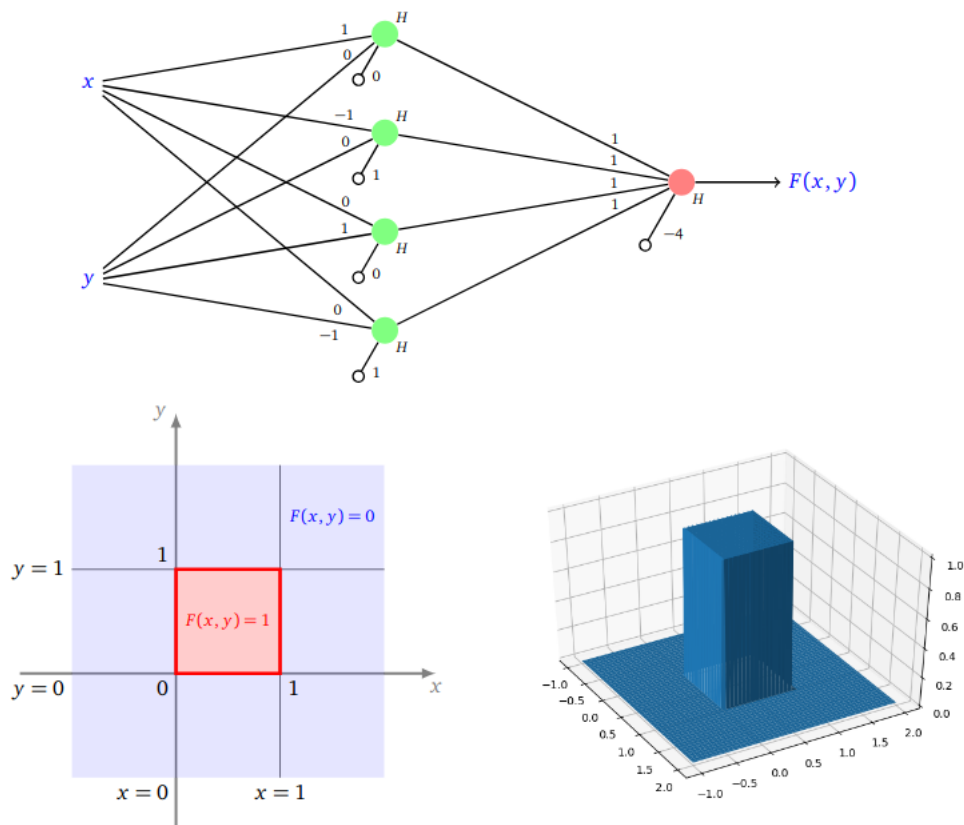
C'est l'**approximation uniforme** des fonctions. La preuve est simple : toute fonction continue sur un intervalle $[a, b]$ peut être approchée d'aussi près que l'on veut par une fonction en escalier. Par exemple, on subdivise l'intervalle $[a, b]$ en n sous-intervalles $[x_i, x_{i+1}]$ et on prend une marche de hauteur $f(x_i)$ sur cet intervalle. Comme nous avons prouvé que l'on sait réaliser toutes les fonctions en escalier, la preuve est terminée.



2.8 Théorème d'approximation universelle (deux variables et plus)

Ce que nous avons fait pour une variable, nous pouvons le faire pour deux variables (ou plus). **Théorème d'approximation universelle** : Toute fonction continue $f : [a, b] \times [a, b] \rightarrow \mathbb{R}$ peut être approchée d'aussi près que l'on veut par une fonction $F : [a, b] \times [a, b] \rightarrow \mathbb{R}$ réalisée par un réseau de neurones.

Il suffit là encore de réaliser des fonctions marches élémentaires. Nous ne donnons pas de détails mais seulement l'exemple d'un réseau qui réalise la fonction $F : \mathbb{R}^2 \rightarrow \mathbb{R}$ qui vaut 1 sur $[0, 1] \times [0, 1]$ et 0 partout ailleurs.



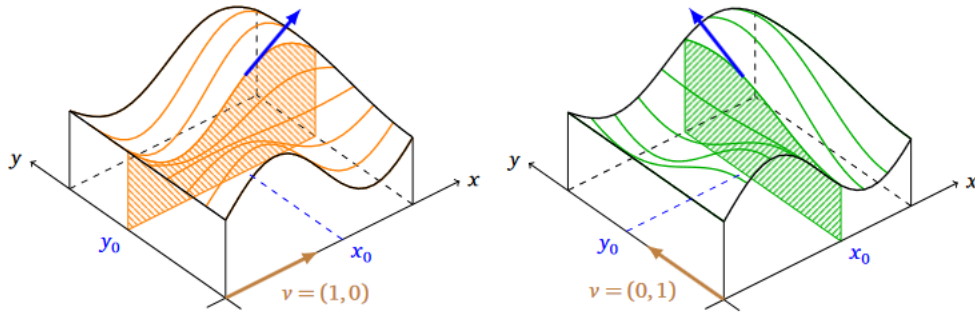
3 Gradient

3.1 Dérivée partielle

Pour une fonction d'une variable, la dérivée est la pente de la tangente au graphe de la fonction (le graphe étant alors une courbe). Pour une fonction de deux variables $(x, y) \rightarrow f(x, y)$, les dérivées partielles indiquent les pentes au graphe de f selon certaines directions (le graphe étant ici une surface). Plus précisément

- $\frac{\partial f}{\partial x}(x_0, y_0)$ est la pente du graph f en (x_0, y_0) suivant la direction de l'axe (Ox) . En effet cette pente est celle de la tangente à la courbe $z = f(x, y_0)$ et est donnée par la dérivée de $x \rightarrow f(x, y_0)$ en x_0 , c'est donc bien $\frac{\partial f}{\partial x}(x_0, y_0)$.
- $\frac{\partial f}{\partial y}(x_0, y_0)$ est la pente du graphe de f en (x_0, y_0) suivant la direction de l'axe (Oy)

Sur la figure de gauche, la dérivée partielle $\frac{\partial f}{\partial x}$ indique la pente de la tranche parallèle à l'axe (Ox) (en orange). Sur la figure de droite, la dérivée partielle $\frac{\partial f}{\partial y}$ indique la pente de la tranche parallèle à l'axe (Oy) (en vert).



3.2 Gradient

Le gradient est un vecteur dont les coordonnées sont les dérivées partielles. Il a de nombreuses applications géométriques car il donne l'équation des tangentes aux courbes et surfaces de niveau. Surtout, il indique la direction dans laquelle la fonction varie le plus vite.

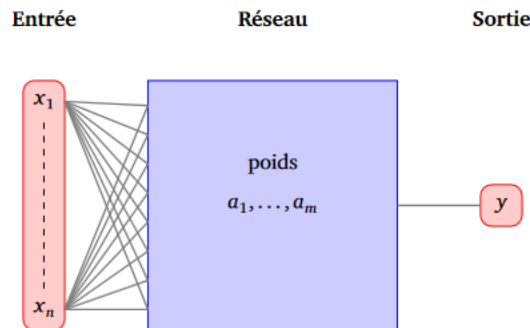
Définition : Le gradient (noté $grad(x_1, \dots, x_n)$ ou $\nabla(x_1, \dots, x_n)$) est tel que pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$, le gradient de f en $(x_1, \dots, x_n) \in \mathbb{R}^n$ est le vecteur :

$$grad(x_1, \dots, x_n) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x_1, \dots, x_n) \\ \dots \\ \frac{\partial f}{\partial x_n}(x_1, \dots, x_n) \end{pmatrix}$$

Exercice : Calculer les gradients de $f(x_1, x_2) = x_1^2 x_2^3$, $f(x_1, x_2, x_3) = x_1^2 \sin(x_2 x_3)$ et $f(x_1, \dots, x_n) = x_1^2 + x_2^2 + \dots + x_n^2$.

3.3 Gradient pour un réseau de neurones

Pour un réseau de neurones $\mathcal{R}$ ayant n entrées $(x_1, \dots, x_n)$ et une seule sortie, nous associons une fonction $F : \mathbb{R}^n \rightarrow \mathbb{R}$, $(x_1, \dots, x_n) \rightarrow F(x_1, \dots, x_n)$. La situation est en fait plus compliquée. Jusqu'ici les paramètres du réseau étaient donnés. À partir de maintenant les variables à dériver ne seront plus les entrées mais les poids du réseau. Notons $a_1, \dots, a_m$ ces poids (ensemble des coefficients et des biais). Si l'entrée est fixée et que les poids sont les variables du réseau alors on pourrait considérer que ce même réseau $\mathcal{R}$ définit la fonction $\tilde{F} : \mathbb{R}^m \rightarrow \mathbb{R}$, $(a_1, \dots, a_m) \rightarrow \tilde{F}(a_1, \dots, a_m)$.



Ce dont nous aurons besoin pour la suite et que nous allons calculer dans ce chapitre, c'est le gradient de $\tilde{F}$ par rapport aux poids $(a_1, \dots, a_m)$, autrement dit, il s'agit de calculer :

$$\frac{\partial \tilde{F}}{\partial a_j}$$

Pour concilier les deux points de vue (entrées et poids), on dira qu'un réseau de neurones ayant des entrées $(x_1, \dots, x_n)$ et des poids $(a_1, \dots, a_m)$ définit la fonction :

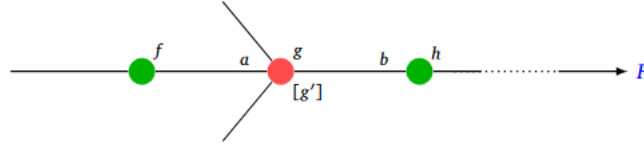
$$\begin{aligned} \hat{F} : \mathbb{R}^n \times \mathbb{R}^m &\rightarrow \mathbb{R} \\ (x_1, \dots, x_n)(a_1, \dots, a_m) &\rightarrow \hat{F}(x_1, \dots, x_n, a_1, \dots, a_m) \end{aligned}$$

Ce n'est pas tout à fait la fonction F dont on voudra calculer le gradient mais notre attention se portera sur une fonction d'erreur E de la forme $E = (F - y_0)^2$, où $\tilde{F}$ est la fonction définie ci-dessus correspondant à une certaine entrée et à la sortie y_0 . On calcule facilement les dérivées partielles de E à partir de celles de $\tilde{F}$ par la formule :

$$\frac{\partial E}{\partial a_j} = 2 \frac{\partial \tilde{F}}{\partial a_j} (\tilde{F} - y_0)$$

3.3.1 Formule du gradient

On considère la portion d'un réseau de neurones :



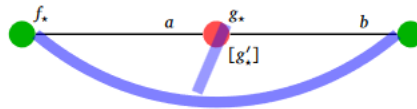
On s'intéresse à une seule arrête entrante du neurone central rouge, celle qui porte le poids a .

- a et b sont des poids
- f , g et h sont des fonctions d'activation,
- f' , g' et h' sont leur dérivées,
- Pour distinguer la fonction de sa valeur en un point on notera f_* est la valeur de la fonction f à la sortie du neurone associé, f'_* est la valeur de la dérivée à la sortie du neurone associé,
- F est la fonction associée au réseau complet

Voici la formule pour calculer la dérivée partielle de F par rapport au coefficient a , connaissant la dérivée partielle par rapport au coefficient b :

$$\frac{\partial F}{\partial a} = f_* \cdot \frac{g'_*}{g_*} \cdot b \cdot \frac{\partial F}{\partial b}$$

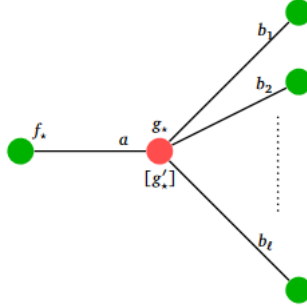
Voici un schéma pour retenir cette "formule du sourire" : on multiplie les coefficients f_* , g'_* , b et la dérivée partielle par rapport à b le long de l'arc, et on divise par le coefficient g_* au bout du segment.



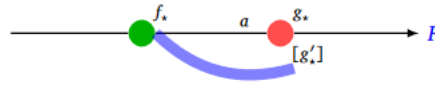
Il est à noter que dans la formule, seule l'arête portant le coefficient a intervient, les autres arêtes entrantes n'interviennent pas (et ne sont pas représentées). Par contre, dans le cas de plusieurs arêtes sortantes il faut calculer la somme des formules précédentes sur chaque arête :

$$\frac{\partial F}{\partial a} = \sum_{i=1}^l f_{\star} \cdot \frac{g'_{\star}}{g_{\star}} \cdot b_i \cdot \frac{\partial F}{\partial b_i}$$

Cette somme comporte autant de termes que d'arêtes sortantes.



Pour pouvoir calculer toutes les dérivées partielles, on procède par récurrence, en partant de la fin puis en revenant en arrière de proche en proche.



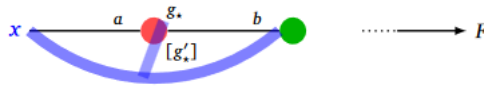
Voici la formule d'initialisation associée au coefficients en sortie de réseau, dite "formule du demi-sourire" :

$$\frac{\partial F}{\partial a} = f_{\star} \cdot g'_{\star}$$

Voici quelques situations particulières, mais qui sont simplement des application de la formule du sourire.

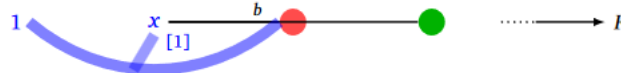
- **Formule à l'entrée.** On applique la formule du sourire avec x à la place de f .

$$\frac{\partial F}{\partial a} = x \cdot \frac{g'_{\star}}{g_{\star}} \cdot b \cdot \frac{\partial F}{\partial b}$$



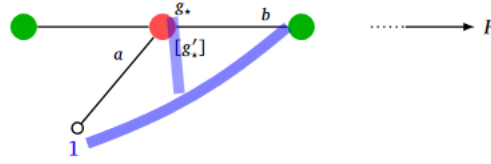
- **Dérivée partielle par rapport aux variables d'entrées.** On applique la formule du sourire en ajoutant des coefficients virtuels égaux à 1 (la dérivée de x par rapport à x est 1).

$$\frac{\partial F}{\partial x} = \frac{b}{x} \cdot \frac{\partial F}{\partial b}$$



- **Cas d'un biais** On applique la formule du sourire en ajoutant un coefficient virtuel égal à 1.

$$\frac{\partial F}{\partial a} = \frac{g'_{\star}}{g_{\star}} \cdot b \cdot \frac{\partial F}{\partial b}$$

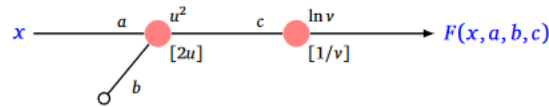


Remarques :

- Ces formules ne sont pas valables lorsque $g_* = 0$. Nous verrons plus loin comment régler ce problème.
- $\frac{\partial F}{\partial a}$ et $\frac{\partial F}{\partial b}$ sont en fait les valeurs des dérivées partielles et ne sont pas vraiment considérées comme des fonctions. Dans la pratique cela ne posera pas de problème car le but est de calculer les valeurs des dérivées partielles (et pas l'expression des fonctions dérivées partielles).

3.3.2 Exemple

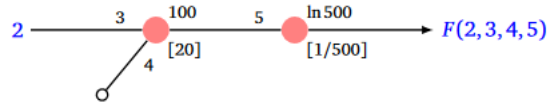
Voici un réseau très simple avec trois poids : a, b, c , une entrée x et une sortie $F(x)$, ainsi que deux fonctions d'activation $u \rightarrow u^2$ et $v \rightarrow \ln v$.



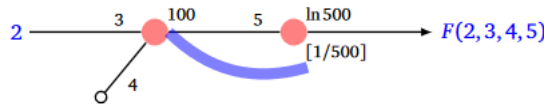
Nous avons l'habitude de considérer la fonction $F(x)$ mais dorénavant les poids seront de nouvelles variables. Nous devons donc étudier la fonction $\hat{F}$ introduite ci-dessous que nous noterons F dans la suite :

$$F(x, a, b, c) = \ln(c(ax + b)^2)$$

Nous souhaitons calculer les dérivées partielles de F par rapport aux poids a, b, c . Nous ne souhaitons pas obtenir une formule générale mais juste la valeur exacte de ces dérivées partielles en un point précis. Nous choisissons l'exemple de $(x, a, b, c) = (2, 3, 4, 5)$. On ré-écrit le réseau avec les valeurs des poids, les valeurs des fonctions et les valeurs des dérivées locales.

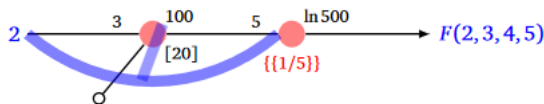


Calcul de la dérivée partielle par rapport à c . On part de la sortie pour l'initialisation. On applique la formule du demi-sourire.



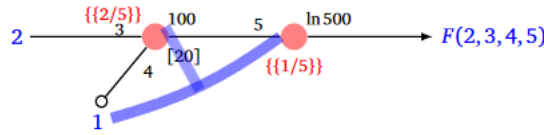
$$\frac{\partial F}{\partial c} = 100 \times \frac{1}{500} = \frac{1}{5}$$

Calcul de la dérivée partielle par rapport à a . On applique la formule du sourire :



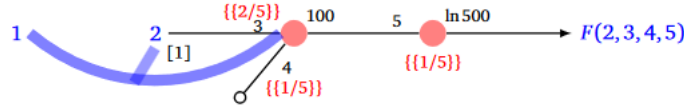
$$\begin{aligned}\frac{\partial F}{\partial a} &= 2 \times \frac{20}{100} \times 5 \times \frac{\partial F}{\partial c} \\ \frac{\partial F}{\partial a} &= 2 \times \frac{20}{100} \times 5 \times \frac{1}{5} \\ \frac{\partial F}{\partial a} &= \frac{2}{5}\end{aligned}$$

Calcul de la dérivée partielle par rapport à b . On applique la formule du sourire (en posant 1 pour le coefficient manquant) :



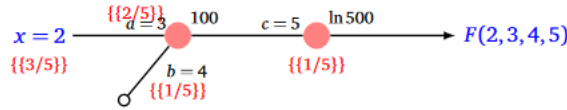
$$\begin{aligned}\frac{\partial F}{\partial b} &= 1 \times \frac{20}{100} \times 5 \times \frac{\partial F}{\partial c} \\ \frac{\partial F}{\partial b} &= 1 \times \frac{20}{100} \times 5 \times \frac{1}{5} \\ \frac{\partial F}{\partial b} &= \frac{1}{5}\end{aligned}$$

Calcul de la dérivée partielle par rapport à x . On peut aussi calculer la dérivée partielle par rapport à x (même si cela ne nous servira pas pour entrainer notre réseau)



$$\begin{aligned}\frac{\partial F}{\partial x} &= 1 \times \frac{1}{2} \times 3 \times \frac{\partial F}{\partial a} \\ \frac{\partial F}{\partial x} &= \frac{3}{5}\end{aligned}$$

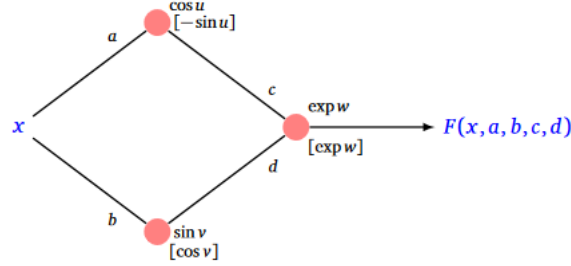
Bilan. Ainsi $F(2, 3, 4, 5) = \ln 500$ et on a calculé les dérivées partielles (entre doubles accolades) pour chacune des variables x, a, b, c



Exercice : Vérifier les formules en calculant directement les dérivées partielles à partir de l'expression $F(x, a, b, c) = \ln(c(ax + b)^2)$.

3.3.3 Second exemple

Pour le réseau précédent, on associe comme précédemment une fonction F .



On considère les poids a, b, c, d comme variables, la fonction $F : \mathbb{R} \rightarrow \mathbb{R}$ s'écrit donc :

$$F(x, a, b, c, d) = \exp(x, a, b, c, d) = \exp(ccos(ax) + dsin(bx))$$

On va noter $u = ax$ et $v = bx$, ainsi $cosu$ et $sinv$ sont les sorties des deux premiers neurones. On note $w = ccosu + dsinv$. La sortie du troisième neurone (qui est aussi la valeur de F) est alors $expw$. On part de la sortie à l'initialisation. Il y a deux dérivées partielles à calculer.

Calcul de la dérivée partielle par rapport à c : On applique la formule du demi-sourire :

$$\frac{\partial F}{\partial c} = cosu.expw$$

Calcul de la dérivée partielle par rapport à d : On applique la formule du demi-sourire :

$$\frac{\partial F}{\partial d} = sinv.expw$$

Calcul de la dérivée partielle par rapport à a : On applique la formule du sourire :

$$\frac{\partial F}{\partial a} = x \frac{-sinu}{cosu} c \frac{\partial F}{\partial c}$$

donc

$$\frac{\partial F}{\partial a} = -xc sinu expw$$

Calcul de la dérivée partielle par rapport à x : Cette dérivée partielle s'obtient comme la somme de deux termes correspondant aux deux arrêtes sortantes :

$$\frac{\partial F}{\partial x} = a \cdot \frac{1}{x} \cdot \frac{\partial F}{\partial a} + b \cdot \frac{1}{x} \cdot \frac{\partial F}{\partial b}$$

donc

$$\frac{\partial F}{\partial x} = (-acsinu + bdcosv)expw$$

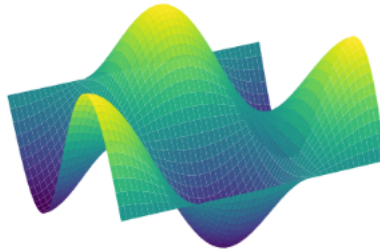
Exercice : Vérifier les formules en calculant directement les dérivées partielles à partir de l'expression $F(x, a, b, c) = \exp(ccos(ax) + dsin(bx))$.

4 Descente de gradient

L'objectif de la méthode de descente de gradient est de trouver un minimum d'une fonction de plusieurs variables le plus rapidement possible. L'idée est très simple, on sait que le vecteur opposé au gradient indique une direction vers des plus petites valeurs de la fonction, il suffit donc de suivre d'un pas cette direction et de recommencer. Cependant, afin d'être encore plus rapide, il est possible d'ajouter plusieurs paramètres qui demandent pas mal d'ingénierie pour être bien choisis.

4.1 Descente de gradient classique

Imaginons une goutte d'eau en haut d'une colline. La goutte d'eau descend en suivant la ligne de plus grande pente et elle s'arrête lorsqu'elle atteint un point bas. C'est exactement ce que fait la descente de gradient : partant d'un point sur une surface, on cherche la pente la plus grande en calculant le gradient et on descend d'un petit pas, on recommence à partir du nouveau point jusqu'à atteindre un minimum local.

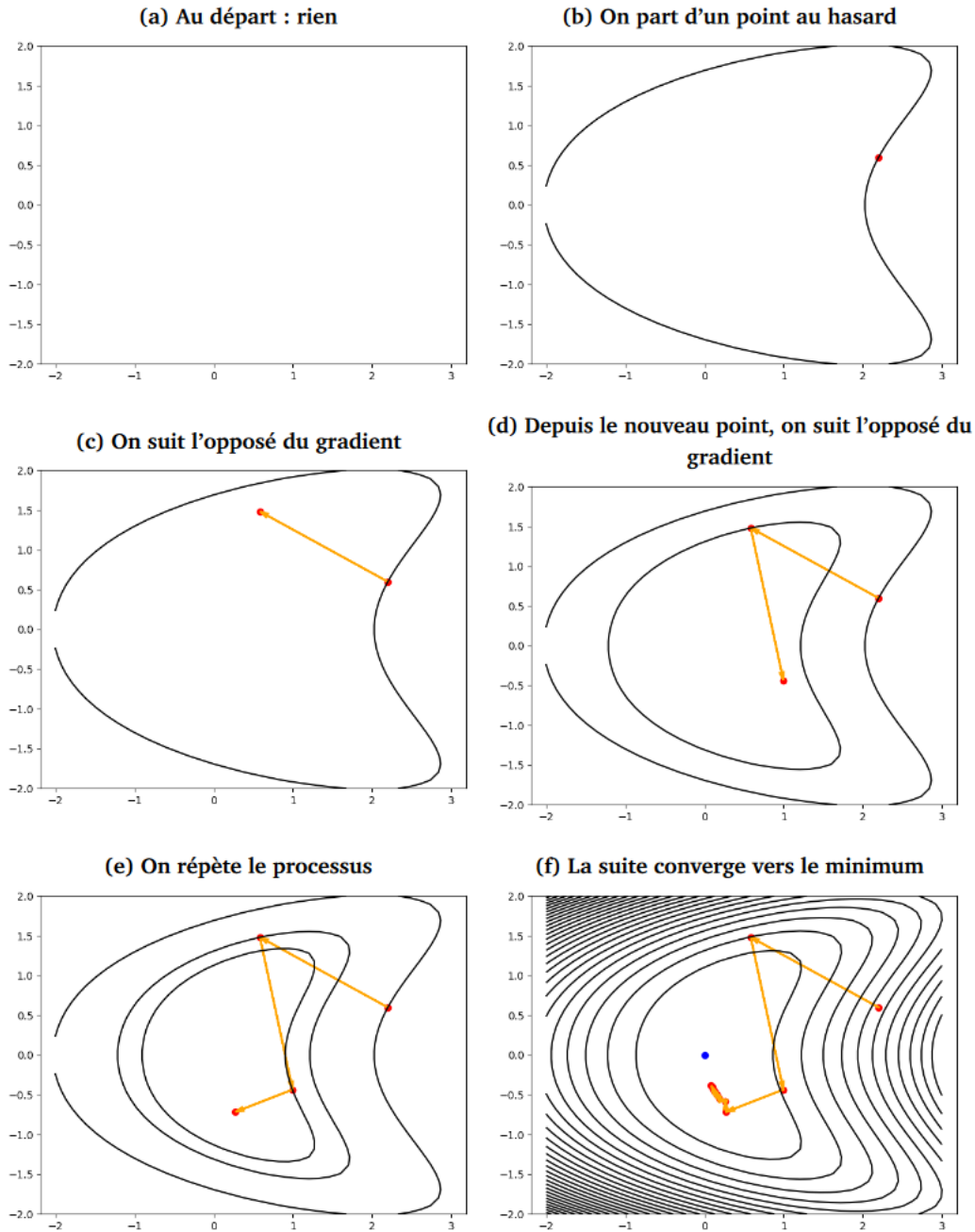


4.2 Rappel sur les lignes de niveaux (fonction à plusieurs variables)

Exercice : Représenter les lignes de niveaux des fonctions $F(x, y) = y - 2x$, $F(x, y) = (x - 1)^2 + y^2$ et $f(x, y) = \exp(y - x^2)$. Les représenter avec Python.

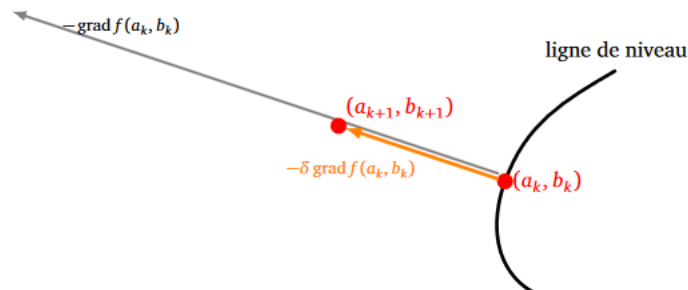
4.2.1 Où est le minimum ?

On nous donne une fonction f de deux variables (a, b) et nous cherchons un point (a_{min}, b_{min}) en lequel f atteint un minimum. Voici la méthode expliquée par des dessins sur lesquels ont été tracées des lignes de niveau.



La seule opération que l'on s'autorisera sera le calcul de $\text{grad}f(a, b)$ en certains points

1. Figure a) : Au départ nous n'avons aucune information globale sur f .
2. Figure b) On choisit un point (a_0, b_0) au hasard. Si on note $c_0 = f(a_0, b_0)$ la valeur de f en ce point, on sait que la ligne de niveau ($f = c_0$) passe par (a_0, b_0) .
3. Figure c) On calcule en ce point le gradient de f . On trace l'opposé du gradient: $-\text{grad}f(a_0, b_0)$. On sait d'une part que la ligne de niveau est orthogonale à ce gradient et surtout que dans la direction de $-\text{grad}f(a_0, b_0)$, les valeurs de f vont diminuer.



On se dirige alors dans la direction opposée au gradient d'un facteur δ (par exemple $\delta = 0.1$). On arrive à un point noté (a_1, b_1) . Par construction, si δ est assez petit, la valeur $c_1 = f(a_1, b_1)$ est plus petite que c_0

4. Figure d) On recommence depuis (a_1, b_1) . On calcule l'opposé du gradient en (a_1, b_1) , on se dirige dans cette nouvelle direction pour obtenir un point (a_2, b_2) où $c = f(a_2, b_2) < c_1$.
5. Figure e) On itère le processus pour obtenir une suite de points (a_k, b_k) pour lesquels f prend des valeurs de plus en plus petites.
6. On choisit de s'arrêter (selon une condition préalablement établie) et on obtient une valeur approchée (a_N, b_N) du point (a_{min}, b_{min}) en lequel f atteint son minimum.

Évidemment avec la vision globale de la fonction, on se dit qu'on aurait pu choisir un point de départ plus près et que certaines directions choisies ne sont pas les meilleures. Mais souvenez-vous que l'algorithme est "aveugle", il ne calcule pas les valeurs de f en (a_k, b_k) et n'a pas connaissance du comportement de f au voisinage de ces points.

4.2.2 Algorithme du gradient

Formalisons un peu les choses pour mettre en évidence l'idée générale et les problèmes techniques qui surviennent :

Algorithme de la descente de gradient :

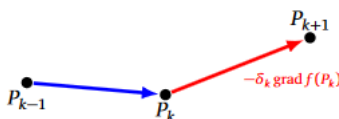
Soit une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $P \rightarrow f(P)$ de plusieurs variables, avec $P = (a_1, \dots, a_n)$ dont on sait calculer le gradient $\text{grad } f(P)$. **Données :**

- Un point initial $P_0 \in \mathbb{R}^n$.
- Un niveau d'erreur $\epsilon > 0$

Itération : On calcule une suite de point $P_1, P_2, \dots \in \mathbb{R}^n$ par récurrence de la façon suivante. Supposons que l'on ait déjà obtenu le point P_k :

- on calcule $\text{grad } f(P_k)$
- on choisit un pas δ_k et on calcule

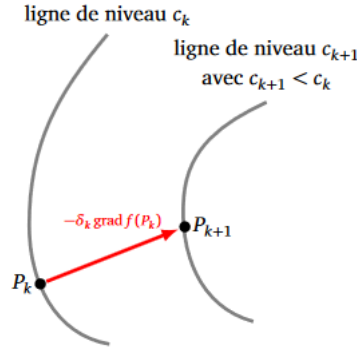
$$P_{k+1} = P_k - \delta_k \text{grad } f(P_k)$$



Arrêt : On s'arrête lorsque $\|\text{grad } f(P_k)\| \leq \epsilon$.

Remarques :

- Évidemment, plus on choisit le point initial P_0 proche d'un minimum local, plus l'algorithme va aboutir rapidement. Mais comme on ne sait pas où est ce minimum local (c'est ce que l'on cherche), le plus simple est de choisir un P_0 au hasard.
- Le choix du pas δ_k est crucial. On sait que l'on peut choisir δ_k assez petit de façon à avoir $f(P_{k+1}) \leq f(P_k)$ car dans la direction de $-\text{grad}f(P_k)$ la fonction f décroît.



On peut également fixer à l'avance un pas δ commun à toutes les itérations, par exemple $\delta = 0.01$. On pourrait également tester à chaque itération plusieurs valeurs δ par balayage ($\delta = 0.001$ puis $\delta = 0.002$, ...) et choisir pour δ_k celui en f qui prend la plus petite valeur.

- Le critère d'arrêt assure qu'en P_k le gradient est très petit. Cela ne garantit pas que ce point soit proche d'un minimum local (et encore moins d'un minimum global). Souvenez-vous : en un minimum local le gradient est nul, mais ce n'est pas parce que le gradient est nul que l'on a atteint un minimum local, cela pourrait être un point-selle voire un maximum local.
- Il est important de calculer $\text{grad}f(a_1, \dots, a_n)$ rapidement. On pourrait bien sûr calculer une approximation de chacune des dérivées partielles $\frac{\partial f}{\partial a_i}(a_1, \dots, a_n)$ comme limite. Mais pour gagner en temps et en précision, on préfère que ce calcul soit fait à l'aide de son expression exacte.

4.3 Optimisation

Nous allons d'une part résoudre des problèmes d'optimisation : quelle droite approche au mieux un nuage de points, et d'autre part étudier comment améliorer le choix du pas δ .

4.3.1 Faire varier le pas

On se concentre d'abord sur le choix du pas δ (*learning rate*). Rappelons tout d'abord que lorsque l'on se rapproche d'un point minimum, le gradient tend vers 0. Le vecteur $\delta \text{grad}f(P_k)$ tend donc vers 0 à l'approche du minimum, même si δ reste constant. Cependant, il faut choisir δ ni trop grand, ni trop petit : δ ne doit pas être trop grand car sinon les points P_k vont osciller autour du minimum, mais si δ est trop petit, alors les points P_k ne s'approcheront qu'au bout d'un temps très long. Une solution est de faire varier δ . Pour les premières itérations, on choisit un δ_k assez grand, puis de plus en plus petit au fil des itérations. Voici différentes formules possibles, à chaque fois δ_0 est le pas initial (par exemple $\delta_0 = 0.1$ ou $\delta_0 = 0.01$).

- **Décroissance linéaire**

$$\delta_k = \frac{\delta_0}{k+1}$$

- **Décroissance quadratique**

$$\delta_k = \frac{\delta_0}{(k+1)^2}$$

- **Décroissance exponentielle**

$$\delta_k = \delta_0 e^{-\beta k}$$

où β est une constante positive.

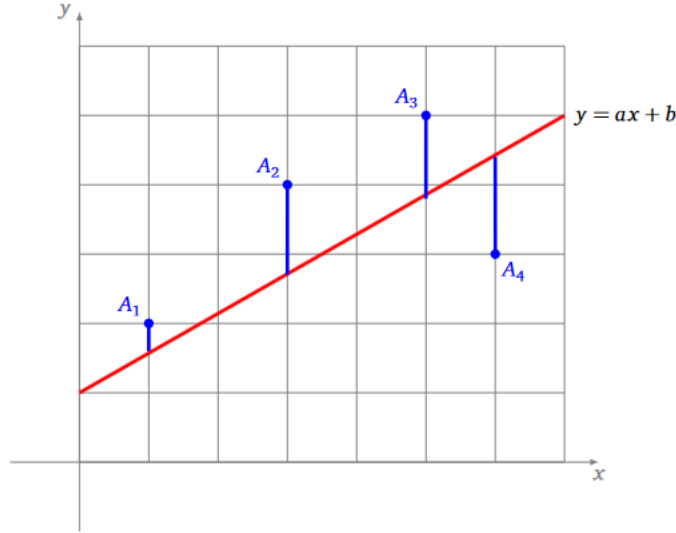
- **Décroissance linéaire utilisée par keras**

$$\delta_k = \frac{\delta_0}{\alpha k + 1}$$

où $\alpha \geq 0$ est une constante (appelée *decay*). Si $\alpha = 0$ alors δ_k est constant (et vaut δ_0). L'usage courant est d'utiliser des valeurs de α entre 10^{-4} et 10^{-6} . Terminons par rappeler que le bon choix d'un δ ou des δ_k n'a rien d'évident, il s'obtient soit par test à la main, soit par des expérimentations automatiques, mais à chaque fois il doit être adapté à la situation.

4.3.2 Régression linéaire $y = ax + b$

On considère un ensemble de N points $A_i = (x_i, y_i)$, $i = 1, \dots, N$. L'objectif est de trouver l'équation $y = ax + b$ de la droite qui approche au mieux tous ces points. Précisons ce que veut dire « approcher au mieux » : il s'agit de minimiser la somme des carrés des distances verticales entre les points et la droite.



La formule qui donne l'erreur est :

$$E(a, b) = \sum_{i=1}^N (y_i - (ax_i + b))^2$$

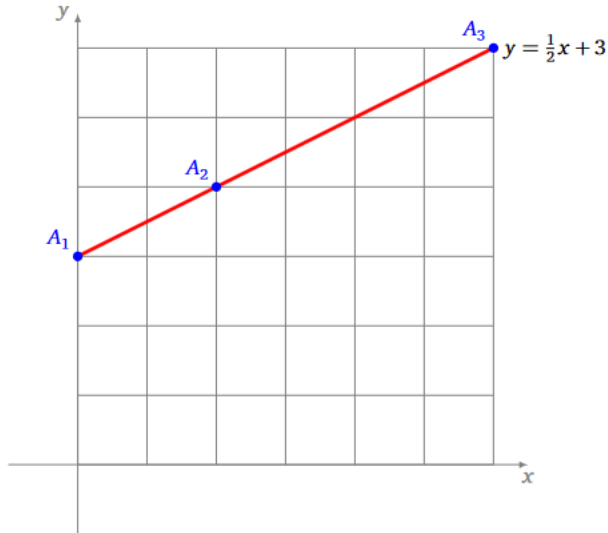
autrement dit

$$E(a, b) = (y_1 - (ax_1 + b))^2 + \dots + (y_n - (ax_N + b))^2$$

Nous allons appliquer la méthode de la descente de gradient à la fonction $E(a, b)$. Pour cela nous aurons besoin de calculer son gradient :

$$\text{grad}E(a, b) = \left(\frac{\partial E}{\partial a}(a, b), \frac{\partial E}{\partial b}(a, b) \right) = \left(\sum_{i=1}^N -2x_i(y_i - (ax_i + b)), \sum_{i=1}^N -2(y_i - (ax_i + b)) \right)$$

On choisit l'exemple trois points $A1 = (0, 3)$, $A2 = (2, 4)$ et $A3 = (6, 6)$ qui sont alignés associés à l'équation $y = \frac{1}{2} + 3$



La fonction $E(a,b)$ s'écrit

$$E(a,b) = (3-b)^2 + (4-(2a+b))^2 + (6-(6a+b))^2$$

Partons arbitrairement de $(a_0, b_0) = (0, 1)$ (qui correspond à la droite horizontale d'équation $y = 1$).

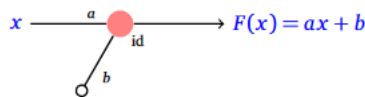
Exercice Calculer les 5 valeurs successives (a_k, b_k) obtenues par la méthode de descente de gradient pour un pas $\delta = 0.02$

Au bout de 100 itérations, on obtient $a_{100} \approx 0.5$ et $b \approx 2.99$ (avec un gradient et une erreur presque nuls). C'est bien la droite $y = \frac{1}{2}x + 3$ qui passe par ces trois points.

Exercice Représenter l'évolution du (a_k, b_k) sur les 5 premières itérations dans les courbes de niveau selon a et b . Dans le plan de coordonnées (x, y) , représenter les droites d'équation $y = a_k x + b_k$ pour les 5 premières valeurs de k .

4.3.3 Régression linéaire et neurone

Quel est le lien entre la régression linéaire et un neurone ? En fait, le neurone suivant a pour fonction associée $F(x) = ax + b$



Reformulons le problème de la régression linéaire ainsi : soit des données (x_i, y_i) , $i = 1, \dots, N$. Il s'agit de trouver pour le type de neurone choisi les coefficients a et b , tels que la fonction F associée au neurone réalise au mieux ces données, c'est-à-dire

$$F(x_i) \simeq y_i$$

Vocabulaire :

- Chaque x_i est une entrée,

- y_i est la sortie attendue pour l'entrée x_i ,
- $F(x_i)$ est la sortie produite,
- $E_i = (y_i - F(x_i))^2$ est l'erreur pour l'entrée x_i
- L'erreur total est

$$E = \sum_{i=1}^N E_i = \sum_{i=1}^N (y_i - F(x_i))^2$$

Vu que $F(x) = ax + b$, on retrouve bien pour E la forme de la fonction présentée précédemment.

4.4 Descente de gradient stochastique

La descente de gradient stochastique (abrégée en *sgd*) est une façon d'optimiser les calculs de la descente de gradient pour une fonction d'erreur associée à une grande série de données. Au lieu de calculer un gradient (compliqué) et un nouveau point pour l'ensemble des données, on calcule un gradient (simple) et un nouveau point par donnée, il faut répéter ce processus pour chaque données.

4.4.1 Petits pas à petits pas

Revenons à l'objectif visé par la régression linéaire. On considère des données (X_i, y_i) , $i = 1, \dots, N$ où $X_i \in \mathbb{R}^d$ et $y_i \in \mathbb{R}$. Ces données proviennent d'observations ou d'expérimentations. Il s'agit de trouver une fonction $F : \mathbb{R}^d \rightarrow \mathbb{R}$ qui modélise au mieux ces données, c'est-à-dire :

$$F(x_i) \simeq y_i$$

Pour l'entrée X_i , sa valeur y_i est la sortie attendue, alors que $F(X_i)$ est la sortie produite par notre modèle. Pour mesurer la pertinence de la fonction F , on introduit la **fonction d'erreur totale** qui mesure l'écart entre la sortie attendue et la sortie produite :

$$E = \sum_{i=1}^N E_i = \sum_{i=1}^N (y_i - F(X_i))^2$$

Cette erreur totale est une somme d'**erreurs locales** :

$$E_i = (y_i - F(X_i))^2$$

Le but du problème est de déterminer la fonction F qui minimise l'erreur E . Par exemple dans le cas de la régression linéaire, il fallait trouver les paramètres a et b pour définir $F(x) = ax + b$. Considérons une fonction erreur $E : \mathbb{R}^n \rightarrow \mathbb{R}$ qui dépend de n paramètres $a_1, \dots, a_n$ (qui définissent l'expression de la fonction F).

Descente de gradient classique. Pour minimiser l'erreur et déterminer les meilleurs paramètres, on peut appliquer la méthode du gradient classique.

On part d'un point $P_0 = (a_1, \dots, a_n) \in \mathbb{R}^n$, puis on applique la formule de récurrence :

$$P_{k+1} = P_k - \delta \text{grad} E(P_k)$$

Pour appliquer cette formule, il faut calculer des gradients $\text{grad} E(P_k)$, or

$$\text{grad} E(P_k) = \sum_{i=1}^N \text{grad} E_i(P_k)$$

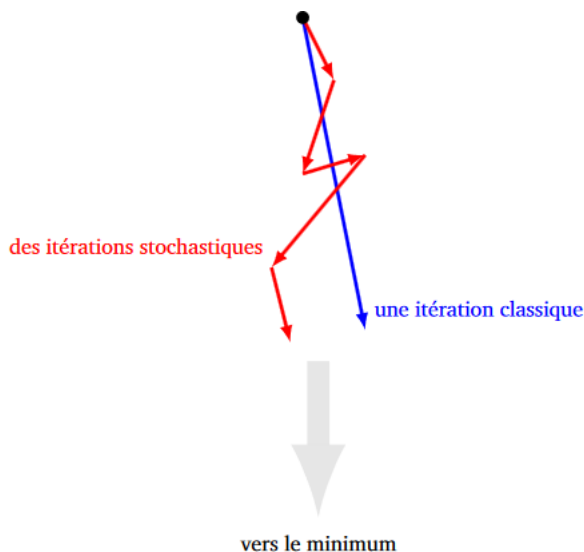
Il faut donc calculer une somme de N termes à chaque itération, ce qui pose des problèmes d'efficacité pour des grandes valeurs de N .

Descente de gradient stochastique

Pour diminuer la quantité de calculs, l'idée est de considérer à chaque itération un seul gradient E_i à la place de E . C'est-à-dire ;

$$P_{k+1} = P_k - \delta \text{grad} E_i(P_k)$$

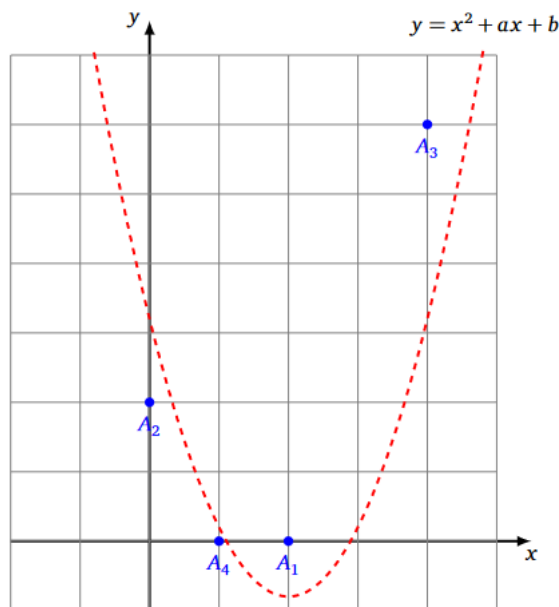
pour *une seule erreur* E_i (correspondant à la donnée numéro i). L'itération suivante se basera sur l'erreur E_{i+1} . Quel est l'intérêt de cette méthode ? Dans la méthode de gradient classique, on calcule à chaque itération un "gros" gradient (associé à la totalité des N données) qui nous rapproche d'un grand oas vers le minimum. Ici on calcule N "petits" gradients qui nous rapprochent du minimum.



Voici les premières itérations de cet algorithme.

- On part d'un point P_0
- On calcule $P_1 = P_0 - \delta \text{grad} E_1(P_0)$. C'est la formule du gradient, mais seulement pour l'erreur locale E_1 (juste à partir de la première donnée (X_1, y_1))
- On calcule $P_2 = P_1 - \delta \text{grad} E_2(P_1)$. C'est la formule du gradient, mais seulement pour l'erreur locale E_2 .
- On itère encore et encore.
- On calcule $P_N = P_{N-1} - \delta \text{grad} E_N(P_{N-1})$. C'est la formule du gradient, mais seulement pour l'erreur locale E_N . À ce stade de l'algorithme, nous avons tenu compte de toutes les données.
- On calcule $P_{N+1} = P_N - \delta \text{grad} E_1(P_N)$. On recommence pour P_N et l'erreur locale E_1 .
- Etc. On s'arrête au bout d'un nombre d'étapes fixé à l'avance ou lorsque l'on est suffisamment proche du minimum.

Exemple On considère les 4 points : $A_1 = (2, 0)$, $A_2 = (0, 2)$, $A_3 = (4, 6)$ et $A_4 = (1, 0)$. Comme ces points ne sont clairement pas alignés, on cherche un modèle pour les placer au mieux sur une parabole.



On va ici chercher des coefficients a et b tels que les points soient proches de la parabole d'équation $y = x^2 + ax + b$. On note donc $F(x) = x^2 + ax + b$ et on souhaite $F(x_i) \simeq y_i$ pour les points $A_i = (x_i, y_i)$, $i = 1, \dots, 4$. Les fonctions d'erreurs locales sont :

$$E_i(a, b) = (y_i - (x_i^2 + ax_i + b))^2$$

La fonction d'erreur globale est :

$$E(a, b) = E_1(a, b) + E_2(a, b) + E_3(a, b) + E_4(a, b)$$

Voici les premières itérations pour chacune des méthodes, la descente de gradient classique (à gauche), la descente de gradient stochastique (à droite) toutes les deux en partant du point $(a_0, b_0) = (1, 1)$ et avec $\delta = 0.01$.

Descente de gradient			Descente de gradient stochastique		
k	(a_k, b_k)	$E(a_k, b_k)$	k	(a'_k, b'_k)	$E(a'_k, b'_k)$
0	(1, 1)	284	0	(1, 1)	284
			1	(0.72, 0.86)	236.43
			2	(0.72, 0.88)	237.41
			3	(-0.38, 0.60)	100.74
1	(-0.54, 0.52)	84.87	4	(-0.40, 0.58)	97.917
			5	(-0.55, 0.50)	83.245
			6	(-0.55, 0.53)	83.913
			7	(-1.22, 0.37)	36.48
2	(-1.36, 0.29)	28.68	8	(-1.22, 0.36)	36.29

Au bout de 200 itérations la descente de gradient classique conduit à $(a_{200}, b_{200}) \simeq (-2.9981, 1.9948)$ (chaque données a été utilisée 200 fois). Cela correspond à 800 itérations de la descente de gradient stochastique (chacune des 4 données a été utilisée 200 fois) cela conduit à $(a_{800}, b_{800}) \simeq (-2.9984, 1.9954)$. La limite cherchée étant $(a, b) = (-3, 2)$, avec $E(a, b) = 0$, les deux méthodes convergent à la même vitesse. Chaque calcul de gradient de la méthode stochastique est très simple mais il faut plus d'itérations.

Exercice Représenter des 5 premiers gradients selon les deux méthodes

Terminons par des remarques plus techniques. Tout d'abord, la formule précise de la descente de gradient stochastique est :

$$P_{k+1} = P_k - \delta \text{grad} E_{(k \% N) + 1}(P_k)$$

où $k \% N$ est "k modulo N" La descente de gradient stochastique est une méthode qui peut être plus efficace :

- Tout d'abord elle n'utilise qu'une donnée à la fois et évite ainsi les problèmes de mémoire de la descente classique pour laquelle il faut manipuler toutes les données à chaque itération.
- Toujours dans le cas où l'on a beaucoup de données, la descente de gradient stochastique peut converger en deux ou trois passages sur l'ensemble des données, alors que la descente classique nécessite toujours plusieurs itérations.
- Avec la méthode stochastique, on calcule des gradients en des points qui sont plus proches du minimum. Attention cependant, certains petits pas peuvent aller dans la mauvaise direction.
- Le caractère aléatoire de ces petits pas est parfois un avantage, par exemple pour s'échapper d'un point-selle

4.5 Différentes fonctions d'erreurs

Il existe différentes formules pour calculer l'erreur entre la sortie attendue y_i et la sortie produite $F(x_i)$. On considère une série de valeurs y_i , $i = 1, \dots, N$ qui sont approchées par des valeurs $F(x_i)$ produites par une formule issue d'un réseau de neurones par exemple. Le but est d'obtenir $F(x_i)$ le plus proche possible de y_i , pour tout $i = 1, \dots, N$. Pour savoir si l'objectif est atteint on mesure l'écart entre ces valeurs.

Erreur quadratique moyenne

$$E = \frac{1}{N} \sum_{i=1}^N (y_i - F(x_i))^2$$

C'est la formule la plus classique (en anglais *minimal squared error* ou mse). Bien entendu $E \geq 0$ quels que soit les $F(x_i)$ et $E = 0$ si et seulement si $y_i = F(x_i)$ pour tous les $i = 1, \dots, N$. C'est presque la formule que l'on a utilisée pour la régression linéaire (il n'y avait pas le facteur $\frac{1}{N}$) **Erreur absolue moyenne**

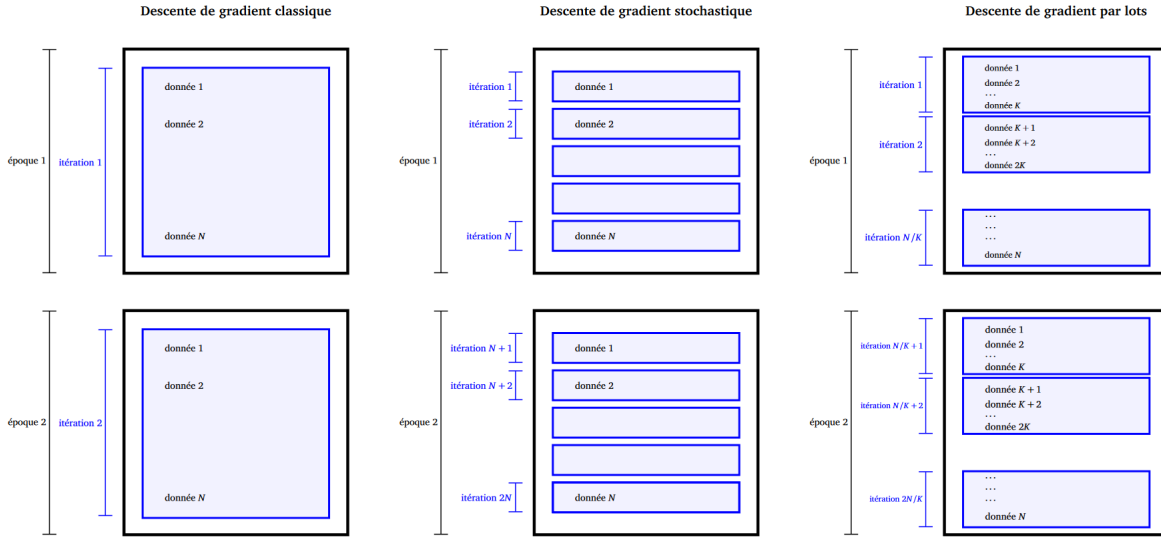
$$E = \frac{1}{N} \sum_{i=1}^N |y_i - F(x_i)|$$

C'est une formule plus naturelle, mais moins agréable à manipuler à cause de la valeur absolue.

Les erreurs locales sont indépendantes les unes des autres, ce qui est la base de la descente de gradient stochastique. Il existe d'autres formules d'erreur, en particulier si la sortie attendue est du type 0 ou 1 ou bien si la sortie produite est une probabilité $0 \leq p \leq 1$.

4.6 Descente par lots

Il existe une méthode intermédiaire entre la descente de gradient classique (qui tient compte de toutes les données à chaque itération) et la descente de gradient stochastique (qui n'utilise qu'une seule donnée à chaque itération). La descente de gradient par **lots** (ou mini-lots, mini-batch) est une méthode intermédiaire : on divise les données par paquets de taille K . Pour chaque paquet (appelé "lot"), on calcule un gradient et on effectue une itération. Au bout de $\frac{N}{K}$ itérations, on a parcouru tout le jeu de données : cela s'appelle une **époque**.



La formule est donc

$$P_{k+1} = P_k - \delta \text{grad}(E_{j_0+1} + E_{j_0+2} + \cdots + E_{j_0+K})(P_k)$$

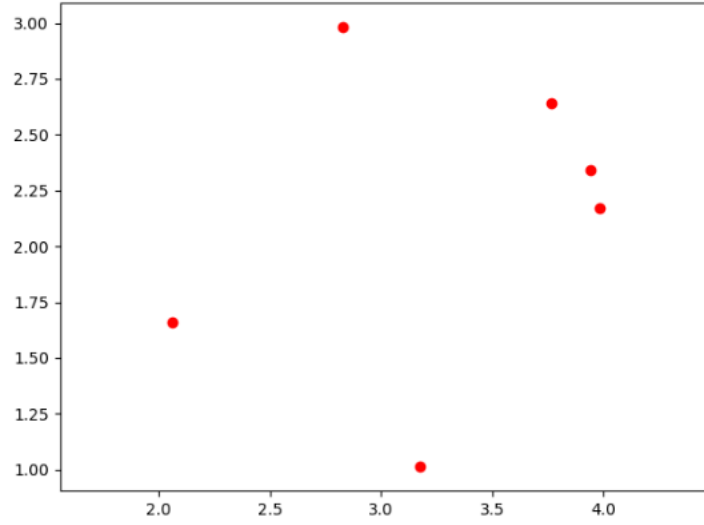
Pour P_{k+2} , on repart de P_{k+1} et on utilise le gradient de la fonction $E_{j_0+K+1} + E_{j_0+K+2} + \cdots + E_{j_0+2K}$.

Remarque :

- Pour $K = 1$, c'est exactement la descente de gradient stochastique. Pour $K = N$, c'est la descente de gradient classique.
- Cette méthode combine le meilleur des deux mondes : la taille des données utilisées à chaque itération peut être adaptée à la mémoire et le fait de travailler par lots évite les pas erratiques de la descente stochastique pure.
- On peut par exemple choisir $2 \leq K \leq 32$ et profiter du calcul parallèle en calculant $\text{grad}(E_1 + \cdots + E_K)$, par le calcul de chacun des $\text{grad}E_i$ sur K processeurs, puis en additionnant les résultats.
- Il est d'usage de mélanger au hasard les données (X_i, y_i) avant chaque époque.

Exemple.

Voyons un exemple d'interpolation circulaire. Les 6 points ci-dessous sont sur un cercle. Comment déterminer son centre et son rayon ?



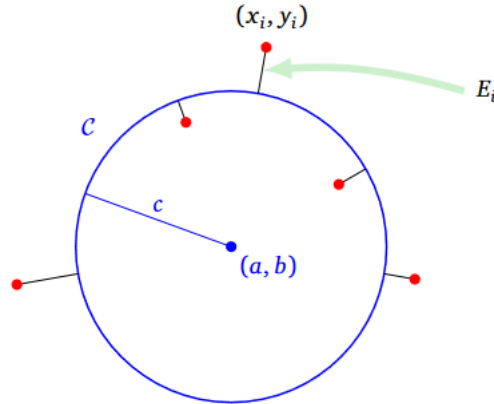
Pour des points (x_i, y_i) , $i = 1, \dots, N$, on mesure la distance globale par rapport au cercle $\mathcal{C}$ de centre (a, b) et de rayon c par la formule d'erreur :

$$E(a, b, c) = \sum_{i=1}^N E_i(a, b, c)$$

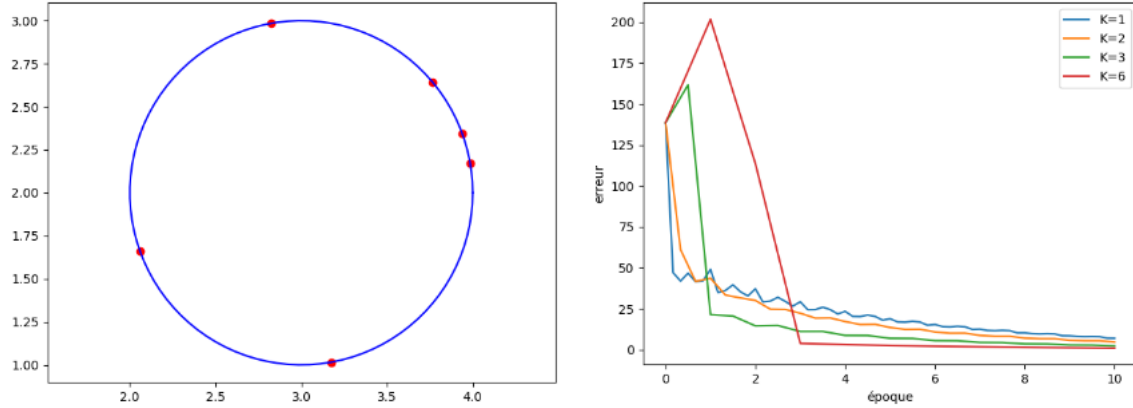
où

$$E_i(a, b, c) = ((x_i - a)^2 + (y_i - b)^2 - c^2)^2$$

En effet, E_i mesure en quelque sorte la distance entre le point (x_i, y_i) et le cercle $\mathcal{C}$ de centre (a, b) et de rayon c . Donc $E_i = 0$ si et seulement si $(x_i, y_i) \in \mathcal{C}$, sinon $E_i > 0$.



Dans notre exemple, les $N = 6$ points sont exactement situés sur le cercle de centre (a, b) et de rayon c . En appliquant la descente de gradient par lots avec $\delta = 0.01$ et $(a_0, b_0, c_0) = (1, 1, 2)$, on trouve $(a, b) = (3, 2)$ et $c = 1$. Ci-dessous, à droite, nous avons représenté les valeurs de l'erreur totale E (qui tend vers 0) en fonction du nombre d'époques et ceci pour différentes tailles du lot $K = 1$ (descente stochastique) $K = 2$, $K = 3$ et $K = 6$ (descente classique).



On remarque qu'au bout de 10 époques la valeur de l'erreur est à peu près la même quelle que soit la taille K de l'échantillon. Par contre, l'évolution au départ est différente. Par exemple pour $K = 1$, l'erreur fluctue à la hausse ou à la baisse à chaque itération. Cette méthode présente bien sûr davantage d'intérêt quand les points ne sont pas exactement sur un cercle. Il s'agit alors de trouver le meilleur cercle qui convient, c'est-à-dire de trouver le minimum (cette fois non nul) de E . Voici un exemple de $N = 100$ points tirés au hasard autour du cercle de centre $(a, b) = (1, 2)$ et de rayon $c = 3$. La descente de gradient est appliquée avec $\delta = 0.001$ et $(a_0, b_0, c_0) = (1, 1, 1)$ pour des lots de différentes tailles $K = 1$, $K = 10$ et $K = 20$. On remarque que deux époques suffisent pour avoir convergence et que plus la taille K de l'échantillon est grande plus la convergence est régulière vers le minimum.

